

Robotics system design, from level 1 to the top

This folder teaches you to design a robot's software as a whole, not just to write one algorithm. It starts with no assumptions: level 1 has no code and no jargon. Each level builds on the one before. Read them in order.

Level	File	What you can do after it
1	What a system is	Draw any robot as boxes and arrows, label the arrows, name a trade-off
2	The robot software stack	Name every box in a real robot's stack, explain topics/services/actions, frames and time
3	Timing, reliability and safety	Build a latency budget, handle failures, draw a state machine, test a robot system
4	Fleets, cloud and scale	Design for 200 robots: what runs where, traffic, updates, monitoring, safety cases
5	Design questions, worked	Take "design a robot that does X" to a full design in 40 minutes, six worked examples

How to study this

- One level per sitting. Draw every diagram yourself on paper before reading the one in the text. Drawing is the skill; reading is not.
- Do the "Try it" exercises. Level 5 contains the answers to the level 1 exercises.
- After level 3, go back to the coding tutorial and notice which box each algorithm lives in: BFS and A* in [tutorial/03](#) are the planner box, the Kalman filter and PID in [tutorial/06](#) are the state estimation and control boxes.

The one idea

Every robot is **sense → think → act**. Designing is drawing those boxes, labelling what flows between them and how often, and choosing trade-offs on purpose. The hard problems live on the arrows: how fresh the data is, how much of it there is, and what happens when it stops.

Level 1. What a system is

This level has no code. Its job is to give you the four words everything else is built on: **component**, **interface**, **data flow**, and **trade-off**. If you can draw a robot as boxes and arrows and say what travels along each arrow, you have finished level 1.

Rough time: one evening.

1.1 A system is boxes and arrows

A system is a set of parts that work together to do one job. To design a system you draw the parts as boxes and the things that pass between them as arrows.

Here is the simplest possible robot system, a robot that follows a black line on the floor:

```

+-----+ +-----+ +-----+
| Sensor | -----> | Brain | -----> | Motors |
| (light  | "how    | (decide | "turn  | (left and |
| sensor) | dark    | which way | left   | right   |
|         | is     | to steer)| a bit" | wheel)  |
+-----+ it") +-----+ +-----+
```

Three boxes, two arrows. That is already a design. Every robot you will ever work on, from this one to a self-driving car, is this same picture with more boxes:

```

SENSE -----> THINK -----> ACT
```

Sense: cameras, lidar, wheel encoders, GPS, microphones. Think: everything that turns sensor data into a decision. Act: motors, arms, grippers, lights, speakers. Keep this triangle in your head. When someone asks you to "design a robot that does X", start by drawing it.

1.2 The four words

Component. A box. It does one job and you can describe that job in a sentence. "Reads the camera and produces images." "Takes a goal and produces a path." A good component can be replaced by a different one that does the same job without the rest of the system noticing. Real example: swapping one lidar for another should only change the lidar driver box.

Interface. The shape of an arrow. It says *what* travels between two boxes and *how often*. "An image, 640 by 480 pixels, 30 times a second." "A path, as a list of (x, y) points, whenever the goal changes." Writing down interfaces is most of system design; the boxes are the easy part. If two people agree on the interface, they can build the two boxes separately.

Data flow. The direction of the arrows, and the order things happen in. Sensor data flows forward towards the motors. Commands flow forward too. Some arrows flow backwards: "the motor reports its actual speed back to the brain". Drawing the flow shows you where the delays are and where a failure will spread.

Trade-off. Every design decision makes one thing better and another thing worse. A faster camera gives fresher pictures but more data to process. A bigger battery lasts longer but makes the robot heavier and slower. There is no design without trade-offs, and saying which trade-off you chose and why is the whole skill. "I chose X because Y matters more than Z here" is the sentence you will say over and over.

1.3 A worked example: a robot vacuum cleaner

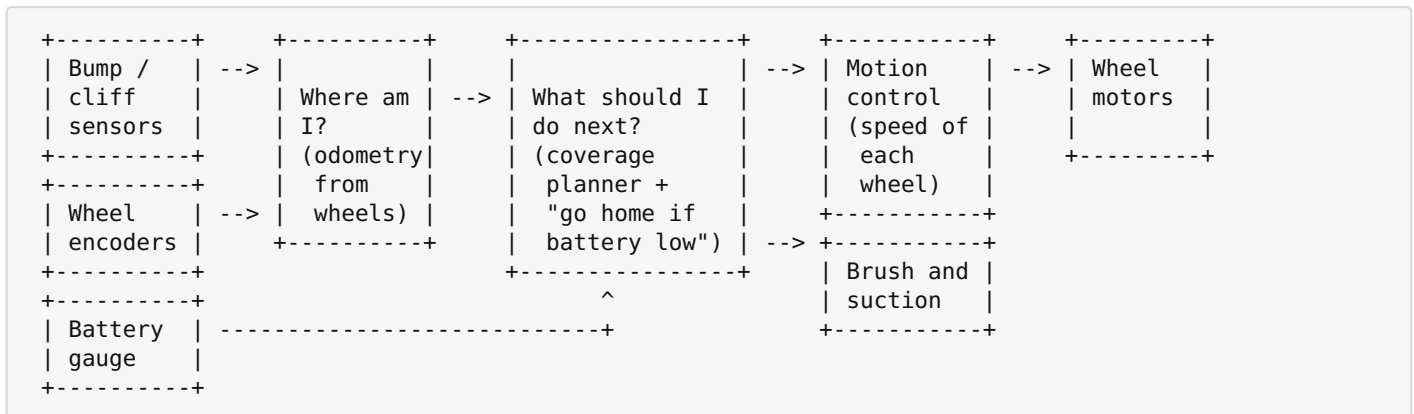
Let us design one with just these four words. The job: clean the floor of a flat without falling down the stairs or getting stuck.

Step 1. What does it sense? Bump sensors (did I hit something), cliff sensors (is there floor under me), wheel encoders (how far have I rolled), a battery gauge. Maybe a small camera or lidar on the expensive models.

Step 2. What does it do? Drive two wheels, run the brush and suction, and go home to charge.

Step 3. What must it think about? Where am I, where have I been, where have I not cleaned yet, am I stuck, is the battery low.

Step 4. Draw the boxes.



Step 5. Write the interfaces. Pick two arrows and say what travels along them:

Arrow	What travels	How often
Wheel encoders → Where am I	two numbers: ticks of the left and right wheel	100 times a second
Where am I → What should I do	the robot's (x, y, heading) estimate	20 times a second
What should I do → Motion control	a target speed and turn rate	20 times a second
Bump sensors → What should I do	"hit something on the left/right/front"	immediately when it happens

Step 6. Name a trade-off. Wheel encoders drift: after ten minutes the robot's idea of where it is can be off by a metre. A lidar fixes that but costs ten times more and needs a faster brain. Cheap models accept the drift and clean in random bounces; expensive models add the lidar and clean in neat rows. Same job, different trade-off, both are valid designs.

That is a complete level-1 system design. Notice that you never wrote code, never named a programming language, and never said "ROS". You drew boxes, named what flows between them, and said one thing you chose and why.

1.4 Three questions to ask about any arrow

Once the boxes are drawn, the interesting problems all live on the arrows. For each arrow ask:

- How fresh is it?** The time between something happening in the world and the box at the end of the arrow knowing about it is called **latency**. A bump sensor with half a second of latency means the robot keeps driving into the wall for half a second. A control loop usually needs latency under a few tens of milliseconds.

2. **How much of it is there?** The amount of data per second is called **throughput** (or bandwidth). A camera at 30 frames a second of 640 by 480 colour pixels is about 28 megabytes a second. Two cameras plus a lidar can saturate a small computer's memory bus. Encoders are a few bytes each.
3. **What happens if it stops?** If the box at the start of the arrow dies or the cable falls out, what does the box at the end do? Keep using the last value? Stop the robot? Switch to a backup? Deciding this for every arrow is what makes a robot safe. The vacuum should stop if the cliff sensor goes silent, because "no reading" might mean "about to fall down stairs".

These three questions, latency, throughput and failure, are the questions a robotics system design conversation is actually about. Level 3 goes deep on them.

1.5 How to draw a diagram that people can read

- Left to right, sense on the left, act on the right.
 - One job per box, and write the job as a verb phrase inside it.
 - Label every arrow with *what* travels on it. An unlabelled arrow is a question you have not answered.
 - Put timing on arrows where it matters ("30 Hz", "on change", "once at startup").
 - Draw failure arrows in a different way, for example dashed, or list them below the diagram.
 - Six to ten boxes fits on a whiteboard. If you need more, group boxes into a bigger box ("Perception") and draw the inside of that box separately.
-

1.6 Try it

Draw, on paper, the boxes and arrows for these three robots. Label every arrow. Name one trade-off for each.

1. A robot arm that picks items from a bin and puts them in a box, using one camera above the bin.
2. A delivery robot that drives on pavements from a shop to a house.
3. A drone that inspects a wind turbine blade for cracks.

Then compare with the answers in level 5, which works these three examples end to end.

1.7 In your own words

A system is components joined by interfaces; the interfaces say what data flows and how often. Every robot is sense, think, act. Designing means drawing the boxes, labelling the arrows, and choosing trade-offs on purpose. The hard questions live on the arrows: latency, throughput, and what happens when an arrow stops.

Level 2. The robot software stack

Level 1 gave you three boxes: sense, think, act. This level opens the "think" box and shows the seven or eight boxes that live inside almost every real robot. Then it shows how those boxes talk to each other (messages and topics), how they agree on where things are (frames) and how they agree on when things happened (time). By the end you can draw a full robot stack from memory and explain every arrow on it.

Rough time: two evenings. There is still no code, but there are links to the code you have already written in the tutorial, because several of these boxes are algorithms you know.

2.1 From three boxes to the standard stack

Start with the picture from level 1:

```
SENSE  ----->  THINK  ----->  ACT
```

Now think about a robot that has to drive from the kitchen to the front door. What does "think" actually have to do, in order?

1. Turn raw sensor bytes into numbers with units. (drivers)
2. Turn those numbers into "there is a chair 2 m ahead". (perception)
3. Work out where the robot itself is. (state estimation, also called localisation)
4. Remember what the world looks like. (mapping)
5. Decide a route to the door. (global planning)
6. Decide how to move in the next second without hitting the chair. (local planning)
7. Turn "move at 0.5 m/s, turn left a bit" into motor currents. (control)
8. Decide what the robot should be doing at all right now: driving, charging, waiting for a lift, stopped because a person is too close. (supervisor, also called the behaviour layer)

Every wheeled robot, arm, drone and self-driving car has these boxes. The names vary a little from company to company, but the jobs do not. Here is each one.

Drivers. A driver is the box that talks to one piece of hardware. It knows the cable, the protocol and the quirks of that one device, and nothing else. In: raw bytes from a USB (universal serial bus) or Ethernet port, or a request to send a command. Out: a clean message with units and a timestamp, for example "a scan of 720 range values in metres, taken at time t". There is one driver per device: lidar driver, camera driver, motor driver, IMU driver. (An IMU is an inertial measurement unit: a chip that measures acceleration and turn rate.) Drivers have no robotics in them. The trade-off they embody is: one tiny box per device because being able to swap a device without touching anything else matters more than saving a box.

Perception. Perception turns sensor data into facts about the world. In: images, lidar scans, point clouds. Out: "objects" with a class and a position ("person at 3.2 m, bearing 10 degrees"), or a cleaned-up scan with the floor removed, or a list of lane lines. Example algorithms: an object detector on images, line fitting and RANSAC (random sample consensus, a way of fitting a line while ignoring outliers) on lidar points (the closest-pair and line-fitting code in [tutorial/06-robotics.md](#), section b4, is a small perception box). Perception is usually the most expensive box in processor time, and the one most likely to be wrong.

State estimation, also called localisation. This box answers "where am I and how fast am I moving". In: wheel encoder counts, IMU readings, a GPS fix if outdoors (GPS is the global positioning system, a satellite position fix good to a few metres), and sometimes matches between the current lidar scan and the map. Out: the robot's pose (position plus heading) and velocity, with a measure of how sure it is. The classic

algorithm is the Kalman filter: the 1D and multi-dimensional versions you wrote in `tutorial/06-robotics.md`, section b1, are exactly this box for a robot moving in a line. The box runs fast, typically 50 to 200 times a second, because everything downstream needs a fresh pose.

Mapping. Mapping remembers what the world looks like. In: the current pose from state estimation and the current scan from perception. Out: a map. The most common map for a ground robot is an occupancy grid: a 2D grid of cells where each cell is "free", "occupied" or "unknown". That is the same `vector<vector<int>>` grid you did BFS on in `tutorial/03-grids-bfs-dfs.md`. Mapping and localisation depend on each other (you need a map to localise, you need a pose to map), and doing both at once is called SLAM, simultaneous localisation and mapping. Mapping can be slow, a few times a second, because the world changes slowly.

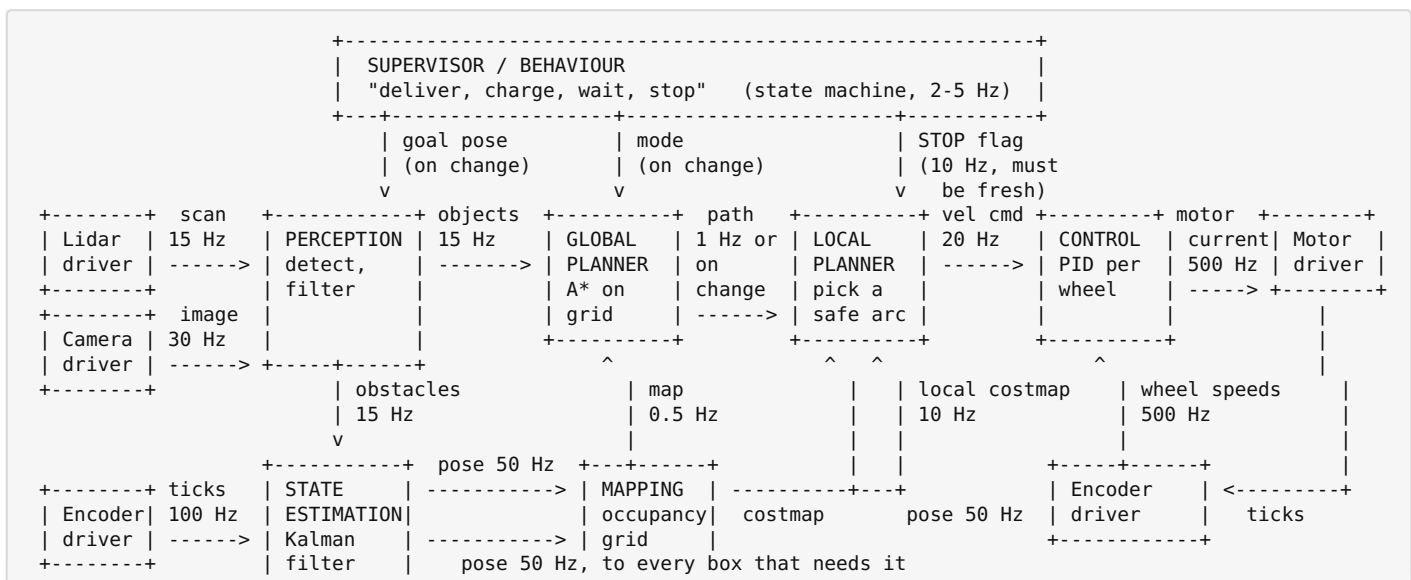
Global planning. Given the map, where I am, and where I want to go, find a route. In: the occupancy grid, start pose, goal pose. Out: a path, a list of (x, y) points from start to goal. Example algorithms: BFS on the grid when every step costs the same (`tutorial/03-grids-bfs-dfs.md`, section A2), Dijkstra when steps have different costs, for example rough ground (`tutorial/04-graphs-dijkstra.md`, section A5), A* when you want the same answer faster by heading towards the goal (`tutorial/03-grids-bfs-dfs.md`, Part C, the project you built). Global planning runs when the goal changes or when the map changes enough, roughly once a second or slower.

Local planning. The global path goes through the chair that appeared after the map was made. Local planning looks only a few metres ahead and a few seconds into the future and picks a safe motion that follows the global path as closely as it can. In: the global path, the current pose and velocity, and a small live map of the nearby obstacles (a local costmap). Out: a velocity command, "forward 0.4 m/s, turn 0.2 rad/s". Example algorithms: try a few hundred candidate arcs, simulate each for two seconds, score by "stays on path, stays away from obstacles, goes fast", pick the best. This runs 10 to 20 times a second.

Control. Control makes the wheels actually do what local planning asked. In: the velocity command, and the measured wheel speeds from the encoders. Out: motor voltages or currents. The classic algorithm is PID, proportional, integral, derivative: the controller with anti-windup you wrote in `tutorial/06-robotics.md`, section b2, is this box for one wheel. Control runs fastest of all, 100 to 1000 times a second, and often on a separate small board, because a slow control loop makes the robot wobble.

Supervisor, also called the behaviour layer. Somebody has to decide what the robot is doing right now. Is it delivering, charging, waiting for the lift, paused because a person stepped in front, or stopped because a sensor died? In: the state of every other box, the battery level, the task list, safety flags. Out: goals for the planner, mode switches, and the master "stop" flag. It is usually a state machine (a diagram of modes and the events that move between them) or a behaviour tree (a tree of "try this, if it fails try that"). It runs slowly, a few times a second, because decisions at this level are about seconds and minutes, not milliseconds.

Here is the whole thing on one page. Read it left to right. Sense is on the left, act is on the right, and the supervisor sits above everything because it talks to all of it.



```

+-----+ imu | |
| IMU | 200 Hz | |
| driver | -----> +-----+
+-----+

```

Two things to notice. First, the rates fall as you move right and up: encoders at 100 Hz, pose at 50 Hz, local planner at 20 Hz, global planner at 1 Hz, supervisor at a few hertz. Fast, small decisions at the bottom; slow, big decisions at the top. Second, one arrow is different from all the others: the STOP flag. It goes from the supervisor straight to control, skipping the planners. Section 2.5 explains why.

A note on what "Hz" means, because it will be on every arrow from now on: hertz is "times per second". 20 Hz is twenty times a second, one every 50 ms.

2.2 Messages, topics and the publish/subscribe idea

The noticeboard

Imagine an office with a noticeboard in the corridor. The lidar driver walks past fifteen times a second and pins a fresh note: "here is my latest scan". It does not know or care who reads it. Perception walks past and reads the latest scan note. So does mapping. So does a logging program that saves everything to disk. If tomorrow someone adds a fourth reader, the lidar driver does not change at all.

```

Lidar driver  ---> | NOTICEBOARD: "scan" | ---> Perception reads
pins a note      | latest note: 720 ranges, time t | ---> Mapping reads
15 times / s     | | | ---> Logger reads
                  +-----+
(writer does not know who reads)      (readers do not know who wrote)

```

This is called publish/subscribe. The writer *publishes*. A reader *subscribes*. The noticeboard is a *topic*. The note is a *message*. The whole point is that the writer and the readers are not joined together: you can replace, add or remove any of them without telling the others. That is the "component" idea from level 1 made real.

This idea is older than any robot framework and exists without ROS: MQTT (message queuing telemetry transport) in the Internet of Things, DDS (data distribution service) in aircraft, Kafka in web backends, all are noticeboards. In robotics the standard vocabulary comes from ROS 2, the Robot Operating System, which is not an operating system but a set of libraries and tools for building robot stacks. Its words are used in almost every UK robotics job advert, so learn them as plain ideas:

ROS 2 word	Plain idea	Example
node	one running program, one box on the diagram	the lidar driver
topic	a named noticeboard	/scan
message	one note, with a fixed layout of fields	720 floats, a frame name, a timestamp
publisher	a node that pins notes on a topic	lidar driver publishes to /scan
subscriber	a node that reads notes from a topic	perception subscribes to /scan

Drawn in ROS 2 words:

```

+-----+ /scan (LaserScan, 15 Hz) +-----+
| lidar_driver | -----> | perception |
| (node) | | | (node) |
+-----+ | +-----+
          | +-----+
          +-----> | mapping |

```

```
| (node) |
+-----+
```

The words in brackets on the arrow, LaserScan, are the message *type*: a fixed layout, "a header with a timestamp and frame name, then a start angle, an angle step, and an array of ranges". Every message on a topic has the same type. Two nodes agreeing on a message type is exactly "agreeing on an interface" from level 1.

The noticeboard has one important property: it is one-way and fire-and-forget. The lidar driver never finds out whether anyone read the scan. That is perfect for sensor data, where the next scan is coming in 66 ms anyway and nobody wants a stale one. It is wrong for some other jobs, which is why there are two more ways to talk.

Request/reply: asking a question and waiting

Sometimes a box needs an answer before it can continue. "Reset the odometry to zero." "What is the current map?" "Is the gripper open?" This is a *service*: one node asks, one node answers, and the asker waits for the answer. Like knocking on a colleague's door and asking a question, rather than pinning a note.

```
+-----+ request: "reset odometry" +-----+
| supervisor | -----> | state estimation |
|             | <----- |             |
+-----+ reply: "done, pose = 0,0,0" +-----+
              (once, when asked; asker waits)
```

The rule for services: use them for things that happen rarely, finish quickly, and where the asker truly needs the answer. Never for a stream of data, and never for anything that takes more than a fraction of a second, because the asker is stuck waiting.

Long-running tasks: ordering food and getting updates

"Navigate to the front door" takes thirty seconds. The supervisor does not want to sit frozen for thirty seconds. It wants to give the order, get "accepted", then get progress ("12 m to go", "8 m to go"), be able to cancel ("a person needs the lift, stop"), and finally get a result ("arrived" or "failed, door blocked"). This is an *action*. It is like ordering food in an app: you place the order, you see "preparing", "on its way", you can cancel before it leaves, and eventually it arrives.

```
+-----+ goal: "go to (12.0, 3.5)" +-----+
| supervisor | -----> | navigation |
|             | <----- | (global + local |
|             | <----- | planner) |
|             | <----- |             |
|             | <----- |             |
|             | <----- |             |
|             | <----- |             |
|             | <----- |             |
|             | <----- |             |
+-----+ (one goal, many feedback notes, one result; asker keeps working)
```

Under the surface an action is built out of the other two: a service to send the goal, a topic for the feedback, a service to fetch the result. You do not need to know that to use it, but it tells you the idea is not magic.

Which one to use

Pattern	Plain idea	Use when	Robot examples
topic	noticeboard, one-way, latest note wins	data that streams, many readers, stale notes are fine to drop	lidar scans, camera images, pose, velocity commands, battery level

Pattern	Plain idea	Use when	Robot examples
service	ask a question, wait for the answer	rare, quick, and the asker needs the reply before continuing	"reset odometry", "save the map", "open the gripper", "get parameter"
action	place an order, get progress, can cancel, get a result	takes seconds or longer, needs progress or cancel	"navigate to goal", "dock and charge", "pick the item in bin 3", "fly the inspection route"

The common mistakes: using a service for sensor data (the reader is stuck waiting and the writer cannot serve two readers), and using a topic for "navigate to goal" (there is no way to learn that it finished, and no way to cancel). If you can say "stream", it is a topic. If you can say "ask", it is a service. If you can say "order and wait for it to arrive", it is an action.

One trade-off to state clearly: a topic drops old notes because freshness matters more than completeness for sensor data. A service never drops a request because completeness matters more than freshness for commands like "save the map". ROS 2 lets you tune this per topic (a setting called quality of service, QoS: "keep the last 1 note" versus "keep all notes and retry"), and level 3 covers it.

2.3 Coordinate frames as a system concern

tutorial/06-robotics.md, section b3, taught you that a frame is a point of view: "the cup is at (1, 0)" means nothing until you say from where. That was one box's problem. In a full stack it becomes everyone's problem, because every box reports positions and no two boxes naturally use the same point of view.

The standard frames on a mobile robot, from the outside in:

```
map -----> odom -----> base_link -----> camera
                                           -----> lidar
                                           -----> imu
```

base_link. The robot's own body. Origin at the centre of the robot, x forward, y left, z up. Every sensor is bolted somewhere on the body, so every sensor frame is a fixed offset from base_link. "The lidar is 30 cm forward and 20 cm up." This offset never changes while the robot is running, so it is published once and remembered.

odom. Where the robot started when it was switched on, according to the wheels and the IMU only. The transform odom to base_link is what state estimation publishes at 50 Hz. It is smooth (it never jumps) but it drifts: after ten minutes it can be a metre off, exactly the vacuum-cleaner trade-off from level 1.

map. A fixed point in the building, the origin of the occupancy grid. The transform map to odom is the correction that localisation computes when it matches a lidar scan against the map: "the wheels think you are at (5.0, 2.0) but the walls say (5.4, 2.1), so shift odom by (0.4, 0.1)". This transform jumps when a correction arrives, and changes slowly, a few times a second.

Why two frames, odom and map, instead of one "where am I"? Because two different boxes want two different things. Control and local planning want a pose that is smooth, with no jumps, so they use odom. Global planning wants a pose that is correct in the long run, so it uses map. Splitting the drift into its own transform (map to odom) lets both be true at once. That is a design decision stated as a trade-off: two frames because smoothness for control matters more than having a single simple answer.

The TF tree as a system of boxes

The set of all frames and the transforms between them is called the TF tree (TF for transform). It is itself built with the noticeboard: every box that knows one transform publishes it, and a small library on the reading side stitches them into a chain.

```

+-----+ map -> odom (2 Hz) +-----+
| localisation | -----> | TF noticeboard |
+-----+ | (topic /tf) |
+-----+ odom -> base_link (50 Hz) +-----+
| state estimation | -----> | any box can ask: |
+-----+ | "where was the camera |
+-----+ | in the map frame at |
+-----+ | base_link -> lidar, | time t?" |
| robot description | base_link -> camera, |
| (published once, | base_link -> imu (static) |
| from a URDF file) | -----> |
+-----+ +-----+

```

(A URDF, unified robot description format, is a file that lists the robot's parts and where each is bolted. Its only job here is to be the source of the static transforms.)

The chain rule from the tutorial, $p_{\text{parent}} = R * p_{\text{child}} + t$ hop by hop, is exactly what the reading-side library does when perception says "person at (3.2, 0.5) in the camera frame" and the planner needs it in the map frame: camera to base_link (static), base_link to odom (from state estimation), odom to map (from localisation), three hops.

Why every sensor message must carry a frame and a timestamp

Look at the question in the diagram: "where was the camera at time t". Not "now". The robot is moving. A person seen in an image taken 80 ms ago must be placed using where the camera was 80 ms ago, not where it is now, or the person is drawn 8 cm from where they really are at 1 m/s (section 2.4 does this arithmetic). So the TF library keeps a short history of every transform, and every sensor message carries two things in its header:

- `frame_id`: which frame the numbers are in ("camera").
- `stamp`: when the measurement was taken.

A sensor box that publishes positions without a frame is publishing numbers with no meaning. A sensor box that publishes without a timestamp forces every reader to guess "probably now", and "probably now" is wrong by exactly the latency of the arrow. The rule for every driver you ever write: stamp the data with the time the sensor measured it, not the time your code received it, and name the frame.

2.4 Time

Every arrow on the stack diagram has a delay. Level 1 called it latency. Once a stack has eight boxes and thirty arrows, time stops being a detail and becomes something you design.

Why 80 ms matters

A camera at 30 Hz exposes for a few milliseconds, reads out, sends the image over USB, the driver unpacks it, perception runs a detector for 40 ms, and the result reaches the local planner. Adding up a typical chain: 33 ms to get the frame out of the camera, 40 ms to detect, 5 ms for the frame transform and fusion, and the local planner runs every 50 ms so on average you wait another 25 and at worst 50. Call it 80 to 130 ms from photon to plan.

At 1 m/s, a walking pace, 80 ms is 8 cm. The robot is 8 cm further along than the image shows. If the robot was turning at 1 rad/s (a brisk turn, about 57 degrees a second), 80 ms is 0.08 rad, 4.6 degrees, and an object 5 m ahead appears 40 cm to the side of where it really is. That is the width of a person. Ignore time and your robot plans around a ghost.

The fix is not "make everything faster", which is expensive and never enough. The fix is: stamp every message with when it was measured, and have each reader use the TF history to ask "where was the robot at that stamp". Then an 80 ms old image is placed correctly, it is just 80 ms old. Latency you know about is a nuisance. Latency you ignore is a bug.

Clocks

"When it was measured" needs a clock. There are usually three, and the mistake is to assume they agree:

- The **sensor's own clock**, if the sensor stamps its data itself (good lidars and cameras do). Accurate for intervals, but it starts from zero when the sensor powers up and drifts.
- The **computer's clock**. What `now()` returns in your C++ code. Also drifts: a cheap crystal at 20 parts per million loses about 72 ms an hour, 1.7 s a day.
- A **shared clock**, when there are several computers on the robot, or a fleet. They are kept together with a protocol such as PTP (precision time protocol) or NTP (network time protocol) so that "t = 1000.250" means the same instant on every machine.

The design rule is: pick one clock as the robot's time, and make every driver convert its sensor's stamps into that clock as the data comes in. A 72 ms per hour drift is nothing for a log file and everything for a control loop. Two boxes on two computers that disagree about "now" by 50 ms will place the same person in two different spots.

Simulated time

When you replay a recorded drive from a log file, or run the stack in a simulator, the clock must be the recording's clock, not the wall clock. Otherwise a message stamped at "t = 1000.250" arrives when the wall clock says three days later, every "how old is this" check fails, and the TF history throws everything away as stale. ROS 2 handles this with a flag (`use_sim_time`) that makes `now()` read from a `/clock` topic instead of the wall clock. The idea exists without ROS too: any stack that wants to be testable in replay needs a single "what time is it" function that can be switched from wall clock to log clock. Design it in from day one; retrofitting it means touching every box.

Pulling 2.3 and 2.4 together: the two things every message in a robot carries are a frame name and a timestamp. Everything else is payload.

2.5 Worked example: a delivery robot on a pavement

The job: carry a bag of shopping from a shop to a house 800 m away, on the pavement, at up to 1.5 m/s, without hitting anyone, and stop safely if anything goes wrong.

Sense. A 2D lidar for obstacles and localisation, a forward camera for recognising people, kerbs and crossings, wheel encoders, an IMU, a GPS receiver for the rough position outdoors, a bumper strip as the last line of defence, and a battery gauge.

Act. Two driven wheels, a lid lock, a small speaker and lights so pedestrians notice it.

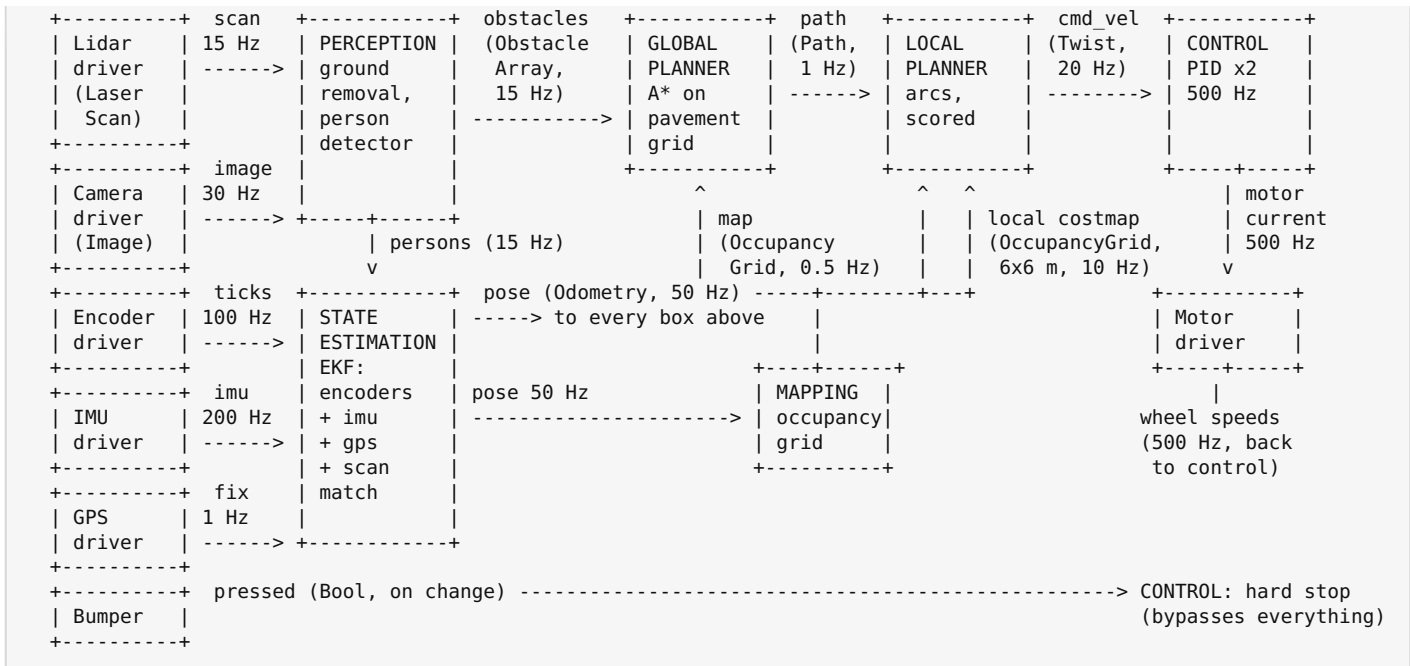
Think. The standard stack from 2.1, plus a GPS input to localisation, plus a supervisor that also talks to a remote operator over a 4G mobile data link when it is stuck.

Here is the full stack. Every arrow has a message type and a rate.

```

+-----+
| SUPERVISOR (state machine, 5 Hz) |
| idle -> loading -> driving -> waiting_at_crossing -> driving -> arrived -> returning |
| any state -> paused (person < 1 m) -> stopped (fault) |
+-----+
| NavigateToGoal | mode | STOP (Bool, 10 Hz, must be | status (String, 1 Hz) |
| (action) | (String, | fresh; missing = stop) | to 4G link |
| v | on change) | v | v |

```



An EKF is an extended Kalman filter, the Kalman filter from the tutorial made to work with turning motion. `Twist` is the message type for "forward speed and turn rate". `Odometry` is "pose plus velocity plus how sure". `OccupancyGrid` is the free/occupied/unknown grid.

The interfaces, written down

Arrow	Message type	Rate	Roughly how much data
Lidar driver → Perception, Mapping, State estimation	LaserScan, 720 ranges	15 Hz	86 kB/s
Camera driver → Perception	Image, 640 by 480 colour	30 Hz	27.6 MB/s
Encoder driver → State estimation	two tick counts	100 Hz	under 1 kB/s
IMU driver → State estimation	3 accelerations, 3 turn rates	200 Hz	12 kB/s
GPS driver → State estimation	latitude, longitude, accuracy	1 Hz	negligible
State estimation → everyone	Odometry (pose + velocity + covariance)	50 Hz	5 kB/s
Perception → Local planner	ObstacleArray (position, size, class "person")	15 Hz	small
Mapping → Global planner	OccupancyGrid, 200 by 200 m at 5 cm cells	0.5 Hz	16 MB per map
Mapping → Local planner	local costmap, 6 by 6 m at 5 cm	10 Hz	288 kB/s
Global planner → Local planner	Path, list of (x, y)	1 Hz or on change	small
Local planner → Control	Twist (v, omega)	20 Hz	negligible
Control → Motor driver	current per wheel	500 Hz	negligible
Supervisor → Navigation	NavigateToGoal action	once per delivery, feedback at 1 Hz	negligible
Supervisor → Control	STOP flag (Bool)	10 Hz, treated as stop if missing	negligible

Arrow	Message type	Rate	Roughly how much data
Bumper → Control	pressed (Bool)	on change	negligible

Two numbers stand out. The camera is 320 times more data than the lidar (27.6 MB/s against 86 kB/s). And the global map is 16 MB, which is why it is sent twice a second and not fifteen times: 15 Hz would be 240 MB/s of mostly unchanged grid.

Three trade-offs, stated properly

1. Lidar plus camera, or camera only? A camera-only robot is cheaper by a few hundred pounds and lighter. But a 2D lidar gives ranges to 0.03 m directly, in the dark and in rain, with 320 times less data to process, and it is what localisation matches against the map. Cameras give class ("that is a person, that is a bin") which a lidar cannot. The delivery robot keeps both: lidar because reliable range in all light matters more than saving the cost of one sensor, and camera because knowing "person" versus "post" changes how wide a berth to give. A camera-only design is valid for a robot that only runs in a well-lit warehouse and can afford a bigger computer.

2. How often to replan globally? At 1 Hz and 1.5 m/s, the robot moves 1.5 m between global plans. That sounds slow, but the global plan is only "which streets and which side of the road"; the local planner at 20 Hz handles everything within 7.5 cm of movement per update. Replanning globally at 10 Hz would cost ten A* runs a second over a 16 million cell grid for no benefit, because the map only changes twice a second anyway. So 1 Hz global, because compute matters more than reacting to a map that has not changed. If the map updated at 10 Hz (say, a busy crossing), the answer would change.

3. Where does the safety stop live? Three candidates. In the supervisor: it knows the most, but it runs at 5 Hz over the general-purpose computer, so between a person appearing and the stop command there can be 200 ms of supervisor period plus all the perception latency. In control: it runs at 500 Hz and is one arrow from the motors, but it knows nothing about people. In the motor driver hardware, wired to the bumper: 1 ms, knows nothing at all. The delivery robot uses all three, in layers. Perception-based slow-down ("person within 2 m, cap speed at 0.5 m/s") lives in the local planner. The STOP flag from the supervisor is checked by control at 500 Hz, and control treats a missing flag as a stop, so a crashed supervisor stops the robot rather than leaving it driving. The bumper goes straight to the motor driver and cuts current with no software in the way. The reasoning: the thing that must never fail should have the fewest boxes between it and the motors, because certainty of stopping matters more than cleverness about when to stop.

Check the arithmetic on that last one: at 1.5 m/s, a stop that lives in a 10 Hz loop adds up to 100 ms, which is 15 cm of travel. A stop in a 1 kHz motor board adds 1 ms, 0.15 cm. And the full stopping distance from 1.5 m/s with a 0.5 s reaction and 2 m/s² braking is 0.75 m of reaction plus 0.56 m of braking, 1.31 m. That is why the local planner starts slowing at 2 m, not at 1 m.

One more number for the report to the shop owner: a 24 V, 40 Ah battery holds 960 Wh; at 150 W average draw (motors, computer, lidar) that is 6.4 hours, about eight deliveries of 800 m each way plus waiting. Level 4 does battery and compute budgets properly.

2.6 How to describe a stack in two minutes

When you need to explain a robot stack to someone, on a whiteboard or in a design document, six sentences cover it. Fill in the brackets.

- The job.** "This robot [does X] in [environment], at up to [speed], and the thing it must never do is [hit a person / drop the load / leave the lane]."
- Sense.** "It senses with [sensors], of which [the lidar] is the one localisation depends on and [the camera] is the one perception depends on."

3. **Where am I.** "State estimation fuses [encoders, IMU, GPS, scan matching] into a pose at [50 Hz]; the map frame corrects the odom frame at [2 Hz]."
4. **Plan.** "A global planner runs [A*] on [an occupancy grid] at [1 Hz]; a local planner picks a safe velocity at [20 Hz] using a [6 m] costmap."
5. **Act.** "Control turns the velocity command into wheel currents at [500 Hz] with [PID]."
6. **Supervise and stop.** "A supervisor state machine at [5 Hz] issues goals as actions and a STOP flag; the STOP is checked in control, missing means stop, and the bumper cuts motor current in hardware, because [certainty matters more than cleverness there]."

Try it on the delivery robot without looking at 2.5. Then try it on your last Python project. If sentence 3 or 6 is empty, that is a gap in the project, not in the template.

2.7 Try it

Draw each of these as the full stack of 2.1, with every arrow labelled with message type and rate, and an interface table. Name three trade-offs for each, in the "X because Y matters more than Z" form. Do the arithmetic for any number you write down.

1. **A warehouse robot** that carries shelves to a packing station on a flat concrete floor with floor markers (small square barcodes) every metre. Questions to settle: is GPS present (no, indoors)? What replaces it for localisation? Does it need a camera at all, or is one downward marker reader enough? What rate does the fleet manager (a computer in the office that hands out jobs) talk to the supervisor at, and is that a topic, a service or an action?
2. **A drone inspecting a wind turbine blade.** The blade is 60 m long, the drone flies 5 m from it at 2 m/s, and a crack is 5 mm wide. Questions: how many pixels across is a crack, and does that set the camera or the distance? Where does the "return home if battery below 30 percent" decision live? The drone has no wheels, so what replaces the encoders in state estimation? How far does the drone move during a 100 ms image latency, and does that matter for a still photo of a crack?
3. **A robot arm picking items from a bin** with one camera above the bin. The arm has six joints, each with its own motor and encoder. Questions: what is the "map" here (hint: it is not an occupancy grid)? What replaces global and local planning (hint: a path is now a list of six joint angles over time)? What does the TF tree look like when the camera is fixed to the ceiling rather than the robot? Which of "detect the item", "plan a grasp", "execute the grasp" is a topic, a service and an action?

Level 5 works all three end to end, so write your answers down before you look.

2.8 In your own words

Every robot's "think" box opens into the same stack: drivers turn hardware into stamped messages; perception turns messages into facts about the world; state estimation says where the robot is; mapping remembers the world; a global planner finds a route on the map; a local planner finds a safe motion for the next second; control makes the motors do it; a supervisor decides what the robot is doing at all. Rates fall as you go up: control at hundreds of hertz, planners at tens, supervisor at a few.

Boxes talk through noticeboards: a publisher pins a message on a topic and any number of subscribers read it, without knowing each other. That is right for streams. For a quick question use a service. For a task that takes seconds and needs progress and cancel, use an action. The words are from ROS 2 but the ideas exist in every robot framework.

Every message carries a frame and a timestamp. The frame says which point of view the numbers are in; the TF tree of published transforms lets any box convert between map, odom, base_link and the sensors. The timestamp says when it was measured, which matters because the robot moved 8 cm during an 80 ms latency at walking pace. One robot clock, converted into by every driver, switchable to a log clock for replay. The safety stop has the fewest boxes between it and the motors, and a missing stop message means stop.

Check yourself

1. Name the eight boxes of the standard stack in order from sensor to motor, and give the typical rate of each. Which two run slowest and why?
2. Which tutorial file holds the code for the state estimation box? For the global planner? For the control box? Name the section in each.
3. A colleague proposes sending lidar scans to perception with a service call. Give two reasons that is the wrong pattern and say what the right one is.
4. "Navigate to goal" is an action. Name the four kinds of message that pass between the supervisor and the navigation box during one action, in order.
5. Why does a mobile robot have both an odom frame and a map frame? Which box publishes each transform, and which frame does the control box read its pose in?
6. A camera message arrives with no `stamp` field. What will every reader assume, and by how much will a person 5 m ahead be misplaced if the robot is turning at 1 rad/s and the true latency is 80 ms? Show the arithmetic.
7. Two computers on a robot disagree about the time by 50 ms. Describe one concrete symptom in the stack diagram of 2.5 and name the fix.
8. What is `use_sim_time` for, and what breaks in the TF library if you replay a log without it?
9. In the delivery robot, the global map is 16 MB and is published at 0.5 Hz. What would the data rate be at 15 Hz, and what is the trade-off sentence for keeping it at 0.5 Hz?
10. Give the three layers of the delivery robot's safety stop, from cleverest to most certain, and the rule for what control does if the STOP message stops arriving.

Level 3. Timing, reliability and safety

Level 1 ended with three questions about every arrow: how fresh, how much, and what if it stops. Level 2 drew the boxes for real robots. This level is about the arrows. By the end you should be able to put a time on every box and add them up, say what the robot does when a box goes quiet, and draw the state machine that decides what the robot is doing right now.

There is still almost no code; the one exception is five lines of C++ in section 3.3.

Rough time: two evenings.

3.1 Latency budgets

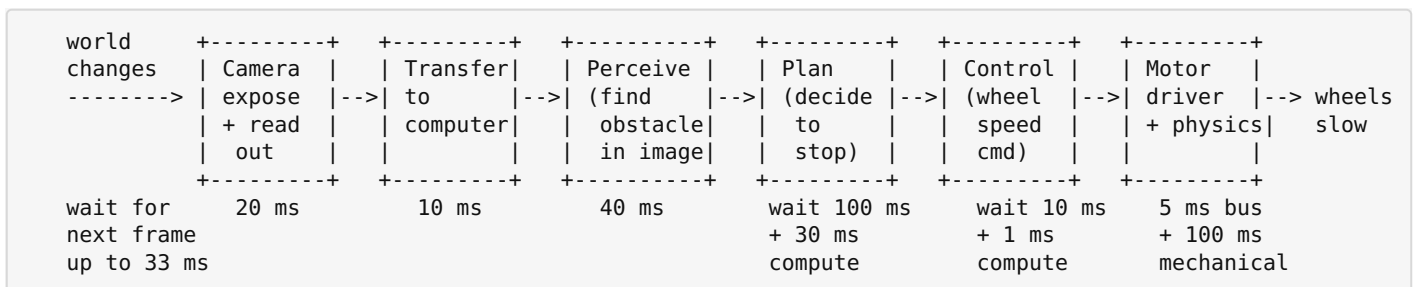
A story

A delivery robot is rolling along a pavement at 1 metre per second. A pushchair is parked 2 metres ahead. How long does the robot have?

The naive answer is "2 seconds". That is wrong twice over. First, the robot cannot stop instantly. If it brakes at 1 metre per second per second (a gentle stop that does not throw the parcels around), it travels half a metre while slowing down. Second, the robot does not know about the pushchair the instant it appears. The camera has to take a picture, the picture has to travel to the computer, a program has to find the pushchair in it, another program has to decide to stop, another has to tell the motors, and the motors have to actually slow the wheels. Every one of those steps takes time, and while they take time the robot keeps rolling at full speed.

The time from "the world changed" to "the wheels start to slow" is the **latency** of the whole chain. A **latency budget** is the list of every step in the chain with the worst time each one is allowed to take, added up, and compared with the time the physics gives you.

The chain, with a time on every box



Two kinds of cost appear on that drawing, and beginners always miss the first kind.

Waiting. The camera runs at 30 hertz (30 times a second, written 30 Hz), so it takes a picture every 33 milliseconds (ms). If the pushchair appears 1 ms after a picture was taken, the next picture is 32 ms away. The worst case is a full 33 ms of waiting before the camera even sees the obstacle. The same happens at every box that runs on its own timer. The planner runs at 10 Hz, so a new obstacle can wait up to 100 ms just for the planner's next tick, even if the planner's actual computation takes 30 ms.

Computing. The time the box spends actually working: running the detector, computing a path.

The budget table

Step	Worst-case time	Why
Wait for the next camera frame	33 ms	camera at 30 Hz, obstacle can appear just after a frame
Camera exposure and readout	20 ms	sensor integrates light, then ships pixels
Transfer to computer	10 ms	USB or Ethernet link
Perception (find the obstacle)	40 ms	neural network or point-cloud clustering
Wait for the next planner tick	100 ms	planner at 10 Hz
Planner compute	30 ms	replan the local path
Wait for the next control tick	10 ms	control loop at 100 Hz
Control compute	1 ms	PID and wheel speed maths
Motor driver and bus	5 ms	command travels over CAN bus to the driver board
Mechanical response	100 ms	motor current ramps, wheels actually start to slow
Total	349 ms	

So the robot travels at full speed for 0.349 seconds after the pushchair appears. At 1 m/s that is 0.35 metres. Then it brakes for another 0.5 metres. Total distance before it stops: 0.85 metres. The pushchair was 2 metres away, so there is 1.15 metres to spare.

Now change one number. Make the robot go 1.5 m/s. The reaction distance becomes 0.52 m and the braking distance becomes 1.13 m (braking distance grows with the square of speed). Total 1.65 m. Still under 2 m, just. At 2 m/s the total is 2.7 m and the robot hits the pushchair. The fastest speed that still stops in 2 m with this chain is about 1.68 m/s.

That is what a latency budget is for. It turns "is the robot fast enough" into a sum on paper, and it shows which box to speed up. The two biggest numbers here are the mechanical response (100 ms) and waiting for the planner (100 ms). Making perception twice as fast saves 20 ms. Running the planner at 20 Hz saves 50 ms. A direct "obstacle very close, stop now" arrow from perception to control, skipping the planner, saves 130 ms. Section 3.9 does that.

What happens when the budget is blown

The budget is a promise: every box finishes within its worst-case time. Boxes break the promise in ways that look harmless in a Python script:

- The perception node writes a big log file to disk and stalls for 500 ms.
- The planner is given a much bigger map than usual and takes 400 ms instead of 30.
- A garbage collector or memory allocator pauses the process for 50 ms at a bad moment.
- The network between two computers on the robot drops a packet and retries for 200 ms.

Take the first one. Perception takes 500 ms once instead of 40 ms. The chain becomes 809 ms. At 1 m/s the reaction distance is 0.81 m, total stopping distance 1.31 m, still under 2 m. At 1.5 m/s the reaction distance is 1.21 m, total 2.34 m, and the robot hits the pushchair. Same robot, same speed that was fine a minute ago, one slow frame.

Three lessons from that:

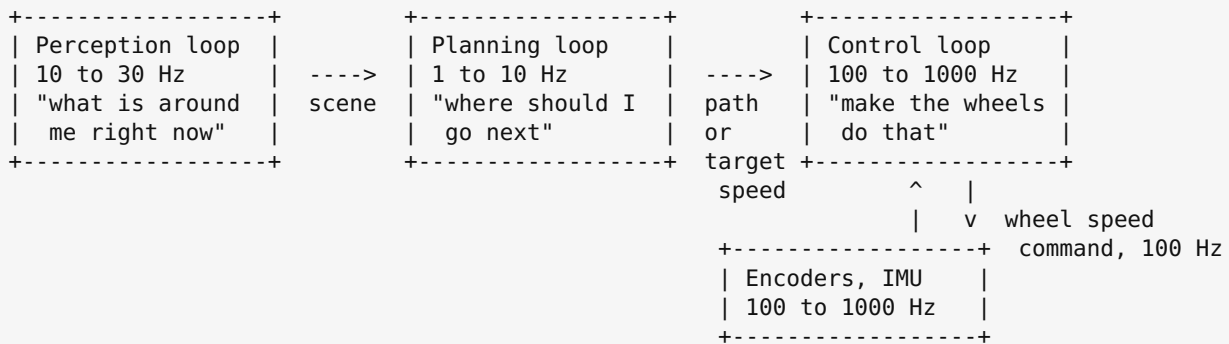
1. A latency budget is about the **worst** case, not the average. A box that is fast on average and slow once an hour is a box that crashes the robot once an hour.
2. The budget decides the **top speed**. If you cannot make the chain faster, you must make the robot slower or give it a sensor that sees further. That is a trade-off you choose on purpose: "top speed 1.2 m/s because stopping distance matters more than delivery time here".

3. Somebody must **watch** for blown budgets at run time, not just at design time. That is what the watchdogs in section 3.4 and the heartbeats in section 3.5 do.

3.2 Loops and rates

What a loop is

Most boxes in a robot run over and over on a timer. Such a box is called a **loop**, and how often it runs is its **rate**, in hertz. The time between two runs is the **period**: 100 Hz means a period of 10 ms, 1000 Hz means 1 ms. A robot has three kinds of loop, and they run at very different rates.



Why the rates are different

Each loop runs as fast as the thing it is fighting changes.

Control loop, 100 to 1000 Hz. It fights physics. A wheel motor's speed can change in a few milliseconds. A quadcopter tilts noticeably in 10 ms. If the control loop looked at the motor only ten times a second, the motor would wander a long way between looks and the correction would arrive late, which makes the robot oscillate. The PID controller in tutorial/06-robotics.md (b2) is this box; it wants a fresh measurement and a new command every period, with as little latency as possible.

Perception loop, 10 to 30 Hz. It fights the sensor. A camera gives 30 frames a second; a lidar spins at 10 or 20 Hz. Faster than the sensor gives nothing new to look at; slower throws frames away. So perception runs at the sensor's rate, or as fast as the computer keeps up.

Planning loop, 1 to 10 Hz. It fights the world, and the world changes slowly compared with a motor. A pedestrian walks about 1.4 m/s; replanning ten times a second means the plan is at most 14 cm stale. A global route across a city can be replanned once a second or only when a road closes. Planning is also expensive (the A* in tutorial/03-grids-bfs-dfs.md over a big grid can take tens of milliseconds), so you run it as rarely as you can get away with.

The trade-off sentence: "the control loop runs at 200 Hz because the motor's time constant is milliseconds and stability matters more than compute cost; the planner runs at 5 Hz because the world does not change faster and compute matters more than a fresher path".

How a slow box talks to a fast box

The control loop ticks every 10 ms. The planner gives it a new target speed every 100 ms. So nine out of every ten control ticks there is no new message. What does the control loop use?

It uses the **latest value**. The planner writes its answer into a slot; the control loop reads the slot every tick and does not care whether the value changed since last time. Think of a whiteboard: the planner writes on it whenever it has something new, the controller glances at it 100 times a second.

LATEST VALUE (a whiteboard)

QUEUE EVERY VALUE (a letterbox)

```

planner ---writes--> [ 0.8 m/s ] <--reads--- control    planner ---puts--> [msg][msg][msg][msg]...
10 Hz                one slot,      100 Hz          10 Hz
                    old value
                    overwritten
                                grows if the reader
                                is slower than the writer
                                --> reader takes from
                                    the front, in order

```

The other way round, a fast box feeding a slow box, is where beginners get hurt. Perception produces a frame at 30 Hz. Suppose the planner consumes frames at 10 Hz, and someone set the arrow up as a queue that keeps every message. Twenty frames a second pile up. After one minute the queue holds 1,200 frames. Each 640 by 480 colour frame is 0.92 megabytes, so that is 1.1 gigabytes of memory. Worse: the frame the planner is looking at after one minute was taken 40 seconds ago. The planner is steering around pedestrians who left long ago.

The rule:

Kind of data	Examples	Arrow style	Why
State: the current value of something	pose, image, lidar scan, battery level, target speed	keep the latest, depth 1	an old state is worthless; only "now" matters
Event: something that happened once	bump, goal reached, "start delivery", emergency stop	queue every one, bounded depth	losing an event means it never happened

Even the event queue must have a **depth** (a maximum length) and a rule for what happens when it is full: drop the oldest, drop the newest, or block the writer. Blocking the writer is the worst choice on a robot because it stalls the fast loop. Dropping the oldest is usual for state; for events you size the queue so it never fills, and log loudly if it does.

In ROS 2 (Robot Operating System 2, the middleware most companies use) this rule is called the **Quality of Service** or QoS setting of a topic: "keep last N" against "keep all", and "reliable" against "best effort". The idea has nothing to do with ROS. Any two loops joined by a buffer have a depth and a drop policy, whether you chose them or a library chose for you.

3.3 Threads and processes for a Python person

The two words

A **process** is a running program with its own private memory. `python planner.py` starts one. If it crashes, only it dies. Two processes cannot see each other's variables; to share data they must send it, over a pipe, a socket, or shared memory.

A **thread** is a worker inside a process. All threads of a process share the same memory: the same variables, the same vectors. Starting a thread is cheap and passing data between threads is free (just a pointer). The price is that two threads can touch the same variable at the same time, which is the bug in this section.

Python threads do not speed anything up because of the Global Interpreter Lock (GIL): only one thread runs Python code at a time. C++ has no GIL; two C++ threads on a four-core computer really do run at the same time. That is why the race below is a real bug in C++ and a rare one in Python.

One node per process, or many nodes in one process

In ROS 2 vocabulary a **node** is one box from your diagram, running as a program. You choose how to pack nodes into processes.

	One node per process	Many nodes in one process
A node crashes	only that node dies; a supervisor restarts it	the whole process dies, every node in it
Passing a 1 MB image	copied, or put in shared memory: 0.1 to 1 ms	pointer handed over: microseconds
Debugging	attach to one small program	one big program, harder to reason about
Typical use	development, non-critical nodes	camera plus perception pipeline, where copy cost matters

The trade-off sentence: "camera and perception share a process because copying 30 frames a second matters more than isolating them; the planner is its own process because being able to restart it after a crash matters more than the copy cost of a path".

Why the control loop gets its own thread

Inside a process, the control loop must tick every 10 ms no matter what. If it shares a thread with the planner, then while the planner is busy for 30 ms the control loop is not running, the motors get no new command, and the robot drifts. So the control loop is given a thread of its own, usually at a higher operating-system priority, and nothing else runs in it.

```
thread A (control, 100 Hz): tick . tick . tick . tick . tick . tick . tick . tick
thread B (planner, 10 Hz):  [===== plan 30 ms =====] . . . . . [===== plan =====]
                           ^ control keeps ticking while B is busy
```

The danger: two hands in one box

Thread B (planner) writes the new target speed into a variable. Thread A (control) reads it. A target speed is a small struct: linear speed and turn rate. Writing two numbers is two operations, and thread A can read in between.

```
shared box: target = { linear, angular }

thread B (writing 0.8, 0.5)          thread A (reading)
-----
writes linear  = 0.8
                                     <--- A reads linear = 0.8, angular = 0.0 (the OLD one)
writes angular = 0.5

A now drives with a pair of numbers that never existed together.
```

This is a **race** (short for race condition): the result depends on which thread got there first. With a two-number struct the symptom is a twitch. With a `std::vector` being resized by one thread while another reads it, the symptom is a crash, or silent garbage, once a week.

Fix one: a lock

A **mutex** (mutual exclusion) is a token that only one thread can hold. Before touching the shared box you take the token; when you are done you give it back. A thread that finds the token taken waits. In C++ you never lock and unlock by hand; you make a `std::lock_guard`, which takes the mutex in its constructor and gives it back in its destructor. That is RAIL, the library-book-that-returns-itself idea from tutorial/02-cpp-interviewers-test.md (m3), and it means the mutex is released on every path, including exceptions and early returns.

```
std::mutex target_mutex;          // one token, shared by both threads
Target target;                   // the shared box
```

```
void set_target(Target t) {           // called by the planner thread
    std::lock_guard<std::mutex> hold(target_mutex); // take the token (constructor)
    target = t;                       // write both numbers together
}                                     // token returned here (destructor)
```

The read side does the same: lock, copy the struct out, unlock, then work on the copy.

The trade-off: locks are simple and correct, but the control thread can be **blocked**. If the planner holds the token for 2 ms because it is doing something silly inside the lock, the control loop's 10 ms tick is 2 ms late. Rule: hold a lock for a copy, never for a computation.

Fix two: a single-writer queue

Give the shared box a direction. Only thread B ever writes; only thread A ever reads; the data goes through a **queue** built so that a writer and a reader never need to wait for each other. For "latest value" data this is a one-slot mailbox that swaps a whole struct in one step. For events it is a lock-free ring buffer of fixed size.

```
planner (only writer) ---> [ slot ] ---> control (only reader)
                           swaps a whole Target at once; no half-written value can be seen
```

The trade-off: the reader never blocks, which is what a control loop wants, but the queue must be built carefully (or taken from a library) and it needs exactly one writer and one reader.

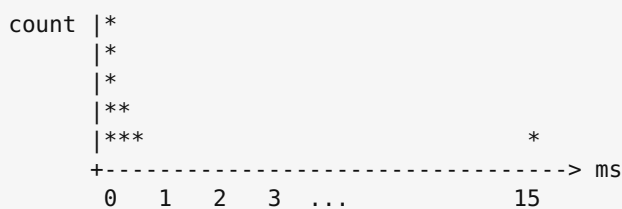
3.4 Real-time in plain words

Fast on average against always on time

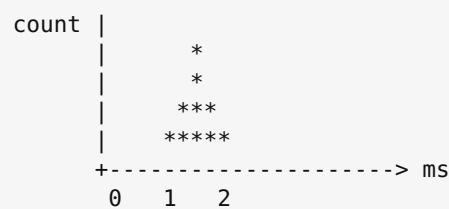
A **deadline** is the moment by which a box must have finished. The control loop at 100 Hz has a deadline every 10 ms. A **real-time** system is one where meeting the deadline is part of being correct: a control tick that arrives at 12 ms is not "a bit slow", it is wrong, because the motor has already drifted.

That is a different goal from "fast". Here are two versions of the same control step:

version 1: average 0.2 ms, worst ever 15 ms



version 2: average 0.9 ms, worst ever 1.1 ms



Version 1 is four times faster on average and useless in a 100 Hz control loop, because once in a while it takes 15 ms and misses its deadline. Version 2 is slower and always on time. For a control loop you want version 2. "Hard real-time" means a missed deadline is a failure (a surgical robot's arm controller, a drone's attitude loop). "Soft real-time" means a missed deadline degrades quality but is survivable (a video stream skipping a frame). Perception and planning are usually soft; control is usually hard.

What is banned inside the control loop

Everything with an unpredictable worst case is banned from the hard loop, even if its average is tiny.

Banned in the control loop	Typical average	Occasional worst case	Do this instead
Heap allocation (<code>new</code> , <code>push_back</code> past capacity, building a <code>std::string</code>)	50 to 500 ns	milliseconds (page fault, allocator lock)	allocate at start-up; <code>reserve()</code> once; use <code>std::array</code> (tutorial/02-cpp-interviewers-test.md, s5)
Taking a lock another thread might hold	20 ns	as long as the other thread holds it	single-writer queue (section 3.3)
Disk or file writes, <code>printf</code> to a console	10 us	10 to 100 ms	push a small record into a queue; a logging thread writes it
Network calls, waiting for a reply	100 us	seconds	never wait in the loop; read latest value
Anything that can throw or retry		unbounded	check inputs before the loop

The `reserve()` note in tutorial/02-cpp-interviewers-test.md (the memory section) and the `std::array` section (s5) are the coding side of this table. A control loop in C++ looks boring on purpose: fixed-size arrays, plain arithmetic, no allocation, no I/O.

A real-time operating system (RTOS), or Linux with the PREEMPT_RT patch, is the other half: it promises that when the 10 ms timer fires, the control thread gets the CPU within tens of microseconds instead of waiting behind a browser update.

Watchdogs

A **watchdog** is a timer that assumes the worst. The brain must reset it ("kick it") every so often, say every 100 ms. If the brain stops kicking, the watchdog fires and does something safe on its own: cuts motor power, engages brakes, or sends a zero-speed command. The watchdog lives in a place that does not depend on the brain: a small microcontroller on the motor driver board, or a hardware timer in the CPU that a crashed program cannot stop.

```
brain (control loop) --- "kick", every 10 ms ---> [ watchdog timer, 100 ms ]
                                     |
                                     no kick for 100 ms? ---> cut motor enable line
```

Numbers: with a 100 ms watchdog, a robot at 1 m/s rolls at most 0.1 m after its brain dies before the motors are cut. At 1.5 m/s with a lazier 300 ms watchdog it rolls 0.45 m. That distance is the price of the watchdog period, so the period is chosen from the physics, not from what is convenient for the software.

The trade-off: a short watchdog period stops the robot sooner but trips on harmless hiccups (a 10 ms watchdog on a loop that occasionally jitters to 11 ms will stop the robot for no reason). "100 ms because 10 cm of roll-on matters less than false stops every few minutes."

3.5 Failure modes

Level 1 asked, for every arrow, "what happens if it stops". Here is the list of things that actually stop, lie or break on a mobile robot, and the two questions for each: how do you notice, and what does the robot do.

What fails	How you detect it	What the robot does
Sensor goes silent (cable, driver crash)	no message for longer than N periods (a timeout on the arrow)	treat as "obstacle everywhere" for safety sensors; stop or slow; try to restart the driver
Sensor lies (lidar sees rain, camera blinded by sun, GPS reflects off buildings)	cross-check with another sensor; value jumps further than physics allows; quality flag from the driver	trust the other sensor more; slow down; if all sensors disagree, stop

What fails	How you detect it	What the robot does
Network drops between computers	heartbeat missing; packet counters; round-trip time rising	each side uses its last good value for a short time, then goes to a safe state
Node crashes	process exit code; missing heartbeat; supervisor notices	supervisor restarts it; if it crashes again within a minute, go to fault state
Battery low or dies	battery gauge below threshold; voltage sag under load	low: go home or finish the current job then stop; critical: stop where safe and call for help
Wheel slips (ice, gravel)	encoders say 1 m/s, IMU and lidar odometry say 0.3 m/s	stop trusting encoders for position; reduce speed; report the location
Motor driver over-temperature	driver reports it on the bus	reduce torque limit; stop before it shuts down on its own
Brain overloaded (CPU at 100 percent)	loop period measured longer than the deadline	shed load: drop perception rate, disable non-critical nodes, slow down
E-stop pressed	hardware line, no software involved	motors cut by hardware; software notices afterwards and enters fault state

Two patterns. Detection is almost always a **timeout** ("I expected a message and did not get one") or a **cross-check** ("two sources disagree"). The response is almost always a move towards a **safe state**.

The safe state

The **safe state** is the state the robot can go to from anywhere, without needing any of the things that might have broken, and in which it can wait for as long as it takes. For a pavement robot it is "stopped, brakes on, lights flashing, sending its position to the operations centre". For a drone it is "hover, then descend slowly", because "stop" is not available. For a surgical robot arm it is "hold position, do not release the tool", because letting go is the dangerous option. For a warehouse robot with a 500 kg shelf on its back it is "stop gently", because a hard stop tips the shelf over.

Choosing the safe state is the first design decision in reliability, and it is different for every robot. Everything else is about getting there.

Heartbeats

A **heartbeat** is a small message a box sends at a fixed rate that says nothing except "I am alive and it is now this time". Its only job is to be missed. Every box that another box depends on sends one; every box that depends on something listens for one and starts a timer.

```

perception ----- heartbeat, 10 Hz -----> supervisor
planner      ----- heartbeat, 10 Hz -----> supervisor
control      ----- heartbeat, 50 Hz -----> supervisor
remote ops   ----- heartbeat, 1 Hz -----> supervisor
supervisor: "no planner heartbeat
for 300 ms? -> command safe stop"
```

Three missed heartbeats at 10 Hz is 300 ms. A robot at 1.5 m/s covers 0.45 m in that time, so the number of misses you tolerate is, again, a physics decision. Heartbeats and watchdogs are the same idea at different layers: a watchdog is a heartbeat whose listener is hardware.

Degraded modes

Between "everything works" and "safe state" there is usually a ladder of **degraded modes**, each one doing less but still useful.

A **state machine** is a drawing with circles and arrows. Each circle is a **state**: a thing the robot can be doing, and in each state exactly one behaviour is active. Each arrow is a **transition**: an event that moves the robot from one state to another. The robot is in exactly one state at any moment, and that state is the answer to "what is the robot doing".

```

+-----+ new job +-----+ arrived +-----+ parcel +-----+ arrived
| IDLE |----->| NAVIGATING |----->| DELIVERING |----->| RETURNING |-----+
+-----+          +-----+ at house +-----+ taken +-----+ at hub |
      ^              ^
      |              |
      |              | crossing
      |              | clear
      |              |
      |              v
      |              |
      |              +-----+
      |              | WAITING AT | (one state, drawn twice to keep
      |              | CROSSING   | the arrows straight)
      |              +-----+
      |              | WAITING AT |
      |              | CROSSING   |
      |              +-----+
      +-----+
any state ---- "fault" (heartbeat lost, e-stop, battery critical) ----> +-----+
                                   | FAULT | --- "operator cleared" ---> IDLE
                                   +-----+

```

From state	Event	To state	What happens on the way
IDLE	new job received	NAVIGATING	load the route; enable the planner
NAVIGATING	road ahead	WAITING AT CROSSING	stop at the kerb; start the crossing checker
WAITING AT CROSSING	crossing clear	NAVIGATING	resume the route
NAVIGATING	arrived at house	DELIVERING	stop; unlock the lid; message the customer
DELIVERING	parcel taken (lid closed, weight sensor zero)	RETURNING	lock the lid; load the route home

From state	Event	To state	What happens on the way
DELIVERING	10 minutes with no customer	RETURNING	lock the lid; flag "not delivered"
RETURNING	road ahead	WAITING AT CROSSING	as above
RETURNING	arrived at hub	IDLE	disable the planner; start charging
any state	fault	FAULT	command safe stop; lights on; report to operations
FAULT	operator cleared	IDLE	re-run self-checks

The rules that make a state machine safe are simple:

- Every state has a way to FAULT, and FAULT needs nothing that might be broken.
- Every transition is logged with a timestamp (section 3.8 depends on this).
- Entering a state resets whatever that state needs; leaving a state cleans up. The "what happens on the way" column is where that lives.
- No state depends on remembering how it was reached. If it does, you are missing a state.

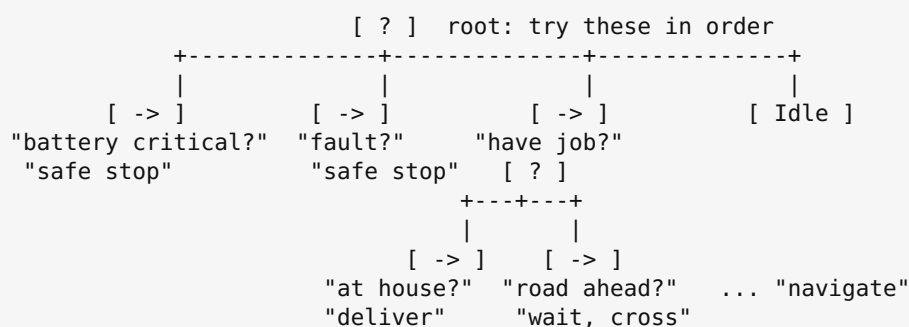
The supervisor ticks slowly, 5 to 10 Hz. It computes no paths and no wheel speeds; it switches other boxes on and off and hands them goals.

When a behaviour tree is better

State machines have a scaling problem. Add a "battery low" check and you need a transition from every state. Add "customer cancelled" and you need another from every state. Ten states and five such conditions is fifty arrows. The table becomes a wall.

A **behaviour tree** is a different way to write the same decisions. It is a tree that is ticked from the root, several times a second. Each node returns one of three answers: success, failure, or running. Two kinds of inner node do the work:

- A **sequence** (drawn `->`) runs its children left to right and stops at the first that fails. "Do A, then B, then C."
- A **fallback** or selector (drawn `?`) runs its children left to right and stops at the first that succeeds. "Try A; if that fails try B; if that fails try C."



Every tick the root asks, left to right: is the battery critical? If yes, safe stop and stop looking. Else, is there a fault? Else, do I have a job? The battery check is written once and applies in every situation, because it is to the left of everything else. That is the thing state machines could not do cheaply.

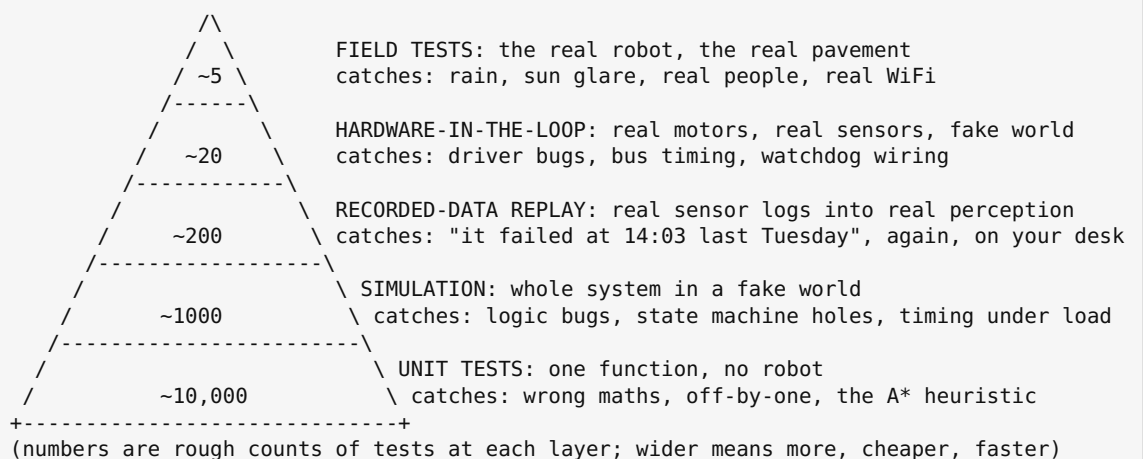
	State machine	Behaviour tree
Best for	a few clear modes with clean hand-overs (delivery robot, vacuum, drone flight phases)	many behaviours that share the same guards (battery, fault, cancel)

	State machine	Behaviour tree
Answer to "what is the robot doing"	the current state, one word	the path from root to the running leaf
Adding a global check	an arrow from every state	one node on the left
Reading it	the table is a complete list of what can happen	you must read tick order to know what wins
Debugging	log the state; easy	log the tree's running path; more to read

The trade-off sentence: "a state machine because the robot has six clear modes and an operations team who need to read the current mode on a dashboard; a behaviour tree would only pay off if the modes multiplied". Many real systems use both: a small state machine on top (idle, working, fault), and a behaviour tree inside "working".

3.7 Testing a robot system

You cannot test a robot by driving it into a pushchair. You can test each layer of the chain in the place where that layer's bugs live cheaply.



Unit tests. One function, known input, expected output, runs in milliseconds. The A* project at `code/03-grids-bfs-dfs/astar` has a `tests/astar_test.cpp` built with GoogleTest: it feeds small grids to the planner and checks the path length and that the path never crosses a wall. That is exactly what a planner box's unit tests look like on a job. Perception boxes are unit-tested with a handful of saved images; controllers with a simulated plant, as in the PID section of `tutorial/06-robotics.md` (b2), which ships with its own tiny simulation.

Simulation. Every box runs for real; only the sensors and motors are replaced by a physics program (Gazebo and Isaac Sim are two common ones). It catches bugs that live between boxes: the state machine never leaves WAITING AT CROSSING; the planner and controller disagree about which way is left; the queue from section 3.2 fills up. It also lets you run a thousand deliveries overnight, with random pedestrians, which the field could never give you. What it does not catch: anything the simulator got wrong, which is always the sensors.

Recorded-data replay. Record every message on every arrow while the real robot drives, with timestamps (in ROS 2 this recording is a **bag**, from the old name rosbag). Later, play the recording back into the perception and planning boxes as if the robot were there. When the robot did something strange at 14:03, you replay 14:02 to 14:04 on your desk, change the code, replay again. Replay also turns one real drive into

a regression test: "the new detector must find every pedestrian the old one found in the 200 hours of bags we have". It cannot test control (the recorded wheels do not respond to your new commands), and it only ever shows you what the robot already saw.

Hardware-in-the-loop (HIL). The real computer, real motor drivers and real sensors, but the world is fake: the motors spin on a bench, the lidar looks at a screen or the simulator injects scans directly at the driver level. It catches the things simulation misses because they are about hardware: the CAN bus stalls when the log rate is high, the watchdog is wired to the wrong pin, the real camera has 60 ms of latency where the simulated one had zero.

Field tests. The real robot on the real pavement, with a person walking behind it holding an emergency stop. It catches what nothing else can: sun glare at 4 pm in October, a dog, WiFi dropping behind one particular building. It is slow and expensive, so everything that could be caught lower in the pyramid should have been.

The trade-off is cost against realism, and the design rule is: push every bug as far down the pyramid as it will go. A bug found in the field should end its life as a unit test or a replay test, so that it never needs the field again.

3.8 Logging and observability

Observability means: when the robot did something, can you find out why afterwards, from what it wrote down. A robot that stops in the street and leaves no trace has failed twice.

What to log, and at what rate

Logging everything is not free. What the delivery robot's sensors produce:

Stream	Size per message	Rate	Per hour	8-hour shift
2D lidar scan (1,080 ranges, 4 bytes each)	4.3 KB	10 Hz	156 MB	1.2 GB
Camera, raw 640 by 480 colour	0.92 MB	15 Hz	50 GB	400 GB
Camera, JPEG compressed (about 50 KB per frame)	50 KB	15 Hz	2.7 GB	22 GB
Odometry (pose, speed, timestamps; about 100 bytes)	100 B	50 Hz	18 MB	0.14 GB
State machine transitions	100 B	on change	KB	KB
Heartbeats and loop timing summaries	50 B	1 Hz per box	MB	MB

Raw camera is out of the question: a shift would fill a laptop. So the rule is layered:

1. **Always, everywhere, cheaply:** every state machine transition, every fault, every timeout, every dropped message, every loop that missed its deadline, with a timestamp and the name of the box. This is tiny and it answers most questions.
2. **Always, at reduced rate:** pose, target speed, battery, lidar scans. Enough to replay what the robot thought was around it.
3. **On trigger only:** the last 30 seconds of compressed camera frames, kept in a ring buffer in memory and written to disk only when a fault, an e-stop, or a hard brake happens. You get the pictures for the interesting moments and nothing for the boring ones.

Everything is stamped with the same clock. If two computers on the robot disagree by half a second, the log is a puzzle instead of a story, so clock sync is part of the design.

"Why did the robot stop at 14:03?"

An operator asks. Here is what the log looks like when the design above has been followed:

```

14:03:11.204 supervisor state NAVIGATING -> FAULT reason: heartbeat timeout: perception
14:03:11.203 supervisor heartbeat missing: perception, last seen 14:03:10.901 (302 ms)
14:03:10.950 control target speed 1.2 -> 0.0 (safe stop commanded by supervisor)
14:03:10.901 perception heartbeat ok, frame 41877, detect 38 ms
14:03:10.860 perception warning: detect took 210 ms (deadline 60 ms), frame 41876
14:03:10.640 perception warning: detect took 190 ms (deadline 60 ms), frame 41875
14:03:10.410 system CPU 98 percent, top: log_writer 61 percent
14:03:10.400 log_writer flushing camera ring buffer (32 MB) after hard-brake trigger at 14:03:09.9
14:03:09.910 control hard brake: decel 2.1 m/s^2 (pedestrian at 0.9 m)

```

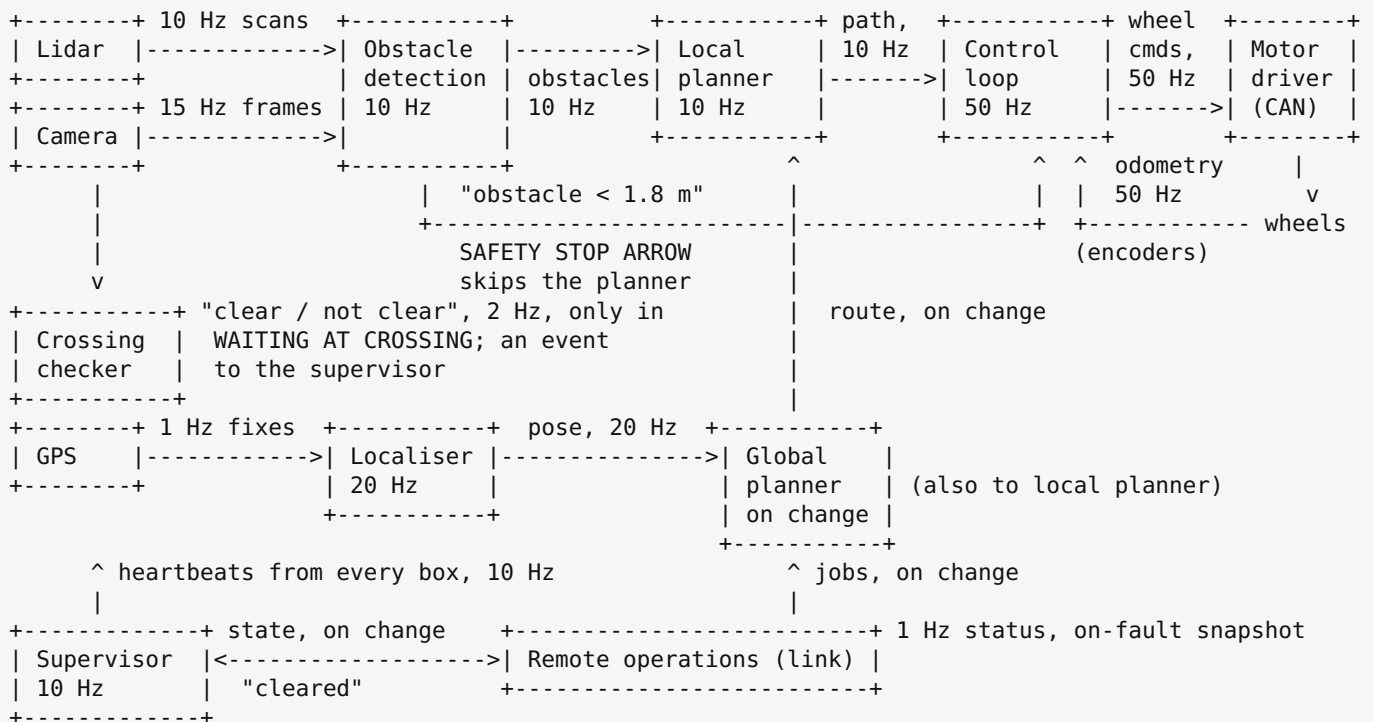
Read from the bottom. A pedestrian stepped out; the robot braked hard (good); the hard brake triggered the camera dump (as designed); the dump ate the CPU; perception missed two deadlines and then a heartbeat; the supervisor went to FAULT as told. Every line came from rule 1 above. The fix is obvious once you can see it: the log writer needs a lower priority than perception. Without the loop-timing warnings and the CPU line you would only know "perception heartbeat lost" and would spend a day blaming the camera driver.

That is observability: enough breadcrumbs, cheaply, that a stop has a story.

3.9 Worked example: the delivery robot, with time and failure on it

Take the Level 2 delivery robot (pavement, shop to house, lidar plus camera plus GPS, a remote operations centre on a mobile link). Level 2 drew its boxes. Now put numbers and failure behaviour on the arrows.

The boxes and their loops



Loop	Rate	Period	Why this rate
Control	50 Hz	20 ms	wheel motors respond in tens of ms; 50 Hz is enough for a slow heavy robot and leaves CPU for perception

Loop	Rate	Period	Why this rate
Obstacle detection	10 Hz	100 ms	the lidar spins at 10 Hz; running faster shows the same scan twice
Local planner	10 Hz	100 ms	pedestrians move 14 cm between plans; enough, and A* over the local grid costs about 20 ms
Localiser	20 Hz	50 ms	fuses odometry (50 Hz) with GPS (1 Hz); 20 Hz keeps the pose fresh for the planner
Global planner	on change		a route across the town only changes when the job or a road closure does
Supervisor	10 Hz	100 ms	checks heartbeats and events; nothing it does needs to be faster
Crossing checker	2 Hz	500 ms	only while stopped at a kerb; a traffic light changes over seconds

Arrows carry "latest value" (scans, frames, pose, path, wheel commands) except three that are queued events: jobs from operations, "cleared" from the operator, and "obstacle < 1.8 m" on the safety arrow, which is an event with a queue depth of 1 (a newer stop overrides an older one, and one is enough).

The latency budget, twice

The normal path, world change to wheels slowing:

Step	Worst case
Wait for the next lidar scan (10 Hz)	100 ms
Lidar transmits the scan	10 ms
Obstacle detection	20 ms
Wait for the next local planner tick (10 Hz)	100 ms
Local planner compute	20 ms
Wait for the next control tick (50 Hz)	20 ms
Control compute	1 ms
Motor driver and CAN bus	10 ms
Mechanical response	150 ms
Total	431 ms

The safety-stop path, which skips the planner:

Step	Worst case
Wait for the next lidar scan	100 ms
Lidar transmits the scan	10 ms
Safety zone check (is any point inside 1.8 m ahead)	2 ms
Wait for the next control tick	20 ms
Control compute	1 ms
Motor driver and CAN bus	10 ms
Mechanical response	150 ms

Step	Worst case
Total	293 ms

Braking at 1 m/s per second (parcels and pedestrians both prefer it), the stopping distances and the fastest safe speed for a given clear distance are:

Speed	Reaction, normal path	Braking distance	Total, normal	Total, safety path
0.5 m/s	0.22 m	0.13 m	0.34 m	0.27 m
1.0 m/s	0.43 m	0.50 m	0.93 m	0.79 m
1.5 m/s	0.65 m	1.13 m	1.77 m	1.57 m

Clear distance ahead	Max speed, normal path	Max speed, safety path
1 m (someone steps out of a doorway)	1.05 m/s	1.15 m/s
2 m (a parked pushchair on a narrow pavement)	1.61 m/s	1.73 m/s
5 m (a wide, open pavement)	2.76 m/s	2.88 m/s

That last table is the design. It says the robot's top speed is not a number but a rule: drive at 1.5 m/s when the lidar shows 2 m or more of clear pavement, and drop to 1 m/s next to doorways and around corners. It also shows that the safety arrow buys only about 0.1 m/s, because the lidar's 100 ms wait and the 150 ms of mechanics dominate both paths. The honest trade-off: "the safety arrow is worth having because it also works when the planner has crashed, not because it is much faster; if we wanted real speed we would buy a 20 Hz lidar, which halves the biggest number in both tables".

Failure handling on this robot

Failure	Detected by	Robot does	Mode
Lidar silent for 300 ms	timeout in obstacle detection	safety stop; supervisor to FAULT	safe state
Camera silent	timeout in detection	continue on lidar at 1.0 m/s; no autonomous crossings	degraded 1
GPS silent or jumping more than 5 m	localiser cross-check with odometry	continue on odometry for 60 s at 0.5 m/s, then stop	degraded 2
Local planner heartbeat lost (300 ms)	supervisor	safety stop; restart the planner; if it dies again within a minute, FAULT	safe state
Control loop heartbeat lost	motor driver watchdog, 100 ms	hardware cuts motor enable; brakes hold	safe state
Mobile link to operations lost	1 Hz heartbeat missed 10 times	continue the current delivery autonomously; do not cross roads; FAULT if still lost after 10 minutes	degraded 1
Battery under 20 percent	gauge	finish the current delivery, return, no new jobs (20 percent of a 4-hour pack is 48 minutes)	degraded
Battery under 5 percent	gauge	stop at the next safe spot; call for pickup	safe state

Failure	Detected by	Robot does	Mode
Wheel slip	encoders against lidar odometry disagree by more than 30 percent	halve speed; stop trusting encoders in the localiser	degraded
E-stop	hardware	motors cut; supervisor to FAULT when it notices	safe state

Numbers behind two rows: three missed 10 Hz heartbeats is 300 ms, and at 1.5 m/s the robot rolls 0.45 m before the safety stop begins, which is why the tolerance is three misses and not ten. The 100 ms hardware watchdog on the control loop costs at most 0.15 m at 1.5 m/s.

The supervisor

The state machine of section 3.6 is this robot's supervisor, plus one thing from the degraded-modes ladder: every state carries a **mode** flag (full, degraded 1, degraded 2) that the speed limit and the crossing checker read. The mode is a flag, not a state, because the robot is still navigating when a camera dies. The trade-off: "a flag because six states times three modes is eighteen states, and the operations dashboard should still read NAVIGATING".

What changed from Level 2

Level 2 drew the same boxes. This level added a rate on every loop, a worst-case time on every box, two budgets turned into a speed rule, a safety arrow that bypasses the planner, heartbeats, a hardware watchdog, a failure table with a safe state and a degraded ladder, and a state machine with FAULT reachable from everywhere. None of it needed code. All of it is the difference between a diagram and a design.

3.10 Try it, in your own words, check yourself

Try it

1. Draw the latency chain for a robot arm that must stop when a hand enters its workspace, watched by a 3D camera at 15 Hz. Put a worst-case time on each box, add them up, and work out how far a 1 m/s arm tip travels before it stops at 5 m/s per second of braking. Is a light curtain (a beam sensor wired straight to motor power) worth adding? What does it do to the chain?
2. Write the failure table for the drone inspecting a wind turbine blade from Level 1's "Try it". Start with the safe state; it is not "stop".
3. Draw the state machine for the bin-picking arm (idle, waiting for image, planning grasp, moving to pick, gripping, moving to place, releasing, fault). Which transitions need a timeout? Then rewrite it as a behaviour tree and decide which drawing you would rather maintain, and why.
4. Run the GoogleTest suite in code/03-grids-bfs-dfs/astar. Add one test for a planner bug you can imagine (the path leaves the grid, say). Which layer of the pyramid did you add to?
5. Change the lidar in section 3.9 to 20 Hz and the mechanics to 80 ms; redo the speed table.

In your own words

Every box takes time, and waiting for a box's next tick is time too. Add the worst cases along the chain, compare with what the physics allows, and you get the robot's top speed. Loops run at the rate of the thing they fight: control at hundreds of hertz for the motors, perception at the sensor's rate, planning at a few hertz for the world. Fast boxes read the latest value from slow boxes; events are queued, with a depth and a drop rule. Threads share memory and therefore need a lock or a single-writer queue; the control loop gets its own thread and avoids anything with an unpredictable worst case, and a hardware watchdog stops the

motors if that thread goes quiet. Every arrow has a timeout and every robot has a safe state, with degraded modes on the way to it. A state machine says what the robot is doing now; a behaviour tree is better when many behaviours share the same guards. Test low in the pyramid, log every transition and every missed deadline, and every stop will have a story.

Check yourself

1. A camera runs at 20 Hz and the box after it runs at 5 Hz. What is the worst-case waiting time added by those two boxes before any computation happens?
2. A robot at 2 m/s has a 400 ms chain and brakes at 2 m/s per second. How far does it travel before stopping? Show the two parts of the answer.
3. Why does the control loop run at 100 Hz or more while the planner runs at 5 Hz? Give the physical reason for each.
4. Perception produces poses at 30 Hz; the logger consumes them at 10 Hz through a queue that keeps every message. Describe what is wrong after two minutes, in memory and in latency.
5. Give one example of state data and one example of event data on a warehouse robot, and say which arrow style each one gets.
6. Two threads share a struct with two fields. Describe the race in one sentence and name the two fixes. Which one would you pick for the control loop's input, and why?
7. What does "real-time" promise that "fast" does not? Name three things banned from a hard control loop and what to do instead of each.
8. Where does a watchdog live, and why there? How would you choose its period for a 2 m/s forklift robot?
9. Name the safe state for a pavement robot, a drone and a surgical arm, and say why they differ.
10. An operator asks why the robot stopped at 14:03. List the four kinds of log line you would expect to find, in the order you would read them.

Level 4. Fleets, cloud and scale

Levels 1 to 3 designed one robot. This level asks what changes when there are two hundred of them. The short answer: the robot stays the same, and a new system grows up around it. That new system has its own boxes, its own arrows, and its own trade-offs, and most of them are about the network, which is the one arrow you cannot trust.

Rough time: two evenings. No code. You need level 1 (boxes, arrows, trade-offs) and the idea from level 3 that every arrow has a latency, a throughput, and a failure mode.

4.1 From one robot to many

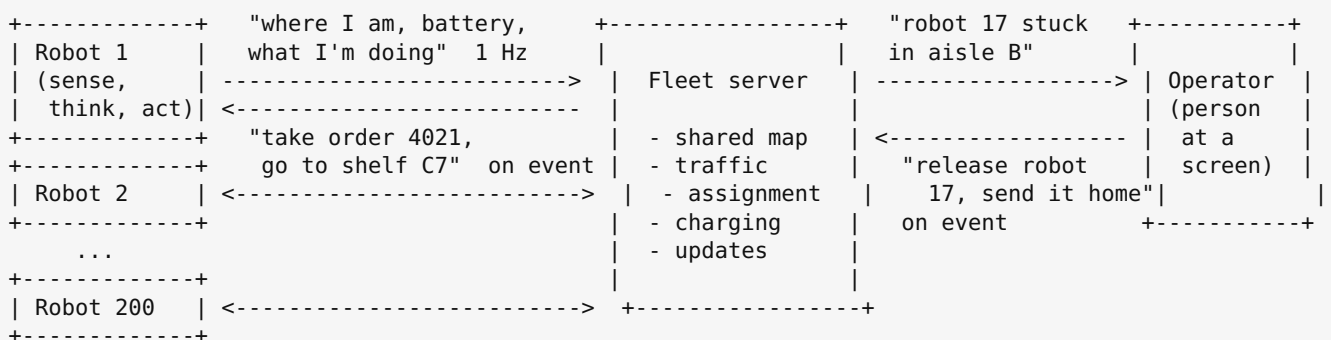
Picture a warehouse the size of two football pitches. Shelves in long rows. One robot drives between the shelves, picks up a tote (a plastic crate of goods), carries it to a packing station, and comes back. You designed that robot in level 2. It has a map, a planner, a controller, a battery.

Now put two hundred of them on the same floor.

Nothing about a single robot's sense, think, act loop changes. But five new problems appear that did not exist with one robot:

1. **Shared map.** Two hundred robots must agree on where the shelves are. If robot 17 sees a fallen tote in aisle B and robot 90 does not know, robot 90 will plan straight through it. The map stops being one robot's private file and becomes something the whole fleet shares.
2. **Traffic.** Two robots that both want the same corridor at the same time will block each other, or worse, deadlock: each waiting for the other to move. With one robot, a path was a path. With two hundred, a path is a claim on space and time.
3. **Task assignment.** Five hundred orders arrive in an hour. Which robot takes which order? The nearest one? The one with the fullest battery? The one that is already going that way?
4. **Charging.** Each robot runs for about eight hours and charges for one. At any moment, roughly one in nine robots is charging. For two hundred robots that is about twenty-two robots on chargers at once, so you need at least that many chargers, plus spare, and a rule for who charges when so they do not all go at the same time.
5. **Updates.** A bug fix that took a USB stick and five minutes on one robot now has to reach two hundred robots, safely, without stopping the warehouse.

Each of these needs a box that is not on any single robot. That box is called the **fleet server** (or fleet manager). It is a computer, usually in the warehouse's server room or in the cloud, that sees all robots at once. Above it sits the **operator**: a person watching a screen who steps in when something goes wrong.



Read the arrows. Up from each robot: a small summary, about once a second. Down to each robot: a task or an instruction, only when something changes. Up to the operator: exceptions, not everything. Down from the operator: decisions, rarely.

The main idea of this level is in that picture. The robot is still a full, self-contained sense, think, act loop. The fleet server does not drive the robot. It tells the robot *what* to do, and the robot works out *how*. Keep that split in your head; every section below is a consequence of it.

4.2 What runs on the robot and what runs in the cloud

Cloud just means computers somewhere else that you reach over a network. It might be a rack in the warehouse office (people say "on-premises" or "on-prem" for that) or a rented machine in a data centre. For design purposes the difference is small. The thing that matters is that there is a network between the robot and the cloud, and networks fail.

Here is the split, with the reason for each row.

Job	Where	Why
Motor control, balance, emergency stop	Robot, always	Needed to stop safely. Must work with the cable unplugged.
Obstacle detection and local avoidance	Robot, always	A person steps out at 1 m. There is no time for a network round trip.
Localisation (where am I on the map)	Robot	Needs sensor data at full rate; sending it away is too much data (see 4.3).
Local path planning (next 10 m)	Robot	Must react to what the robot sees now.
Map building	Either	On the robot when exploring alone; in the cloud when merging 200 robots' views into one map.
Global route across the warehouse	Cloud, usually	The server knows where every other robot is, so it can avoid traffic. The robot can fall back to its own A* if the server is silent.
Task assignment (which robot takes which order)	Cloud	Needs to see all robots and all orders at once. One robot cannot.
Charging schedule	Cloud	Same reason: it is a fleet-wide decision.
Analytics (which aisle causes the most stops)	Cloud	Needs weeks of data from all robots. Not time critical.
Software updates	Cloud sends, robot installs	The robot must be able to say "not now, I am carrying a tote".

The rule that generates the whole table:

Anything the robot needs in order to stop safely must not depend on the network.

State it as a trade-off, in the level-1 sentence: local avoidance runs on the robot because a safe stop matters more than having the cleverest possible planner. Task assignment runs in the cloud because seeing the whole fleet matters more than working when the network is down, and a robot with no task simply waits, which is safe.

A useful test for any box: unplug the network cable and ask what the robot does in the next two seconds. If the answer is "keeps driving on stale commands", the box is in the wrong place.

4.3 Communication: the arrow you cannot trust

Wi-Fi, 4G and 5G in plain words

Wi-Fi is a short-range radio link to a box on the wall called an **access point**. Range is tens of metres indoors. A warehouse has many access points, and a moving robot hops from one to the next. Each hop can drop the connection for a fraction of a second to several seconds. Wi-Fi is what you use indoors because you own the access points and pay nothing per byte.

4G and **5G** are the mobile phone networks. Range is kilometres, you do not own the towers, and you pay per gigabyte. 4G gives a robot on a pavement a few megabits per second uploading, with a round trip (send a message and get a reply) of roughly 50 to 100 milliseconds on a good day. 5G is faster and can have a shorter round trip, but coverage on UK streets is patchy, so you design for 4G and treat 5G as a bonus. Outdoors you use 4G or 5G because you cannot put an access point on every lamp post.

Both kinds of link have two properties you design around:

- **Bandwidth**: how many bits per second you can push through. Shared with everyone else on the same access point or tower.
- **Dropouts**: periods of seconds to minutes where nothing gets through at all. Behind a metal shelf, inside a lift, under a railway bridge. These are normal, not faults.

Why you send summaries, not raw sensor data

Do the arithmetic. Level 1 said one colour camera at 640 by 480 pixels, 30 frames a second, is about 28 megabytes a second. In bits, that is:

$$640 \times 480 \text{ pixels} \times 3 \text{ bytes} \times 30 \text{ frames/s} = 27.6 \text{ MB/s} = 221 \text{ Mbit/s}$$

A single warehouse Wi-Fi access point shares about 100 Mbit/s between everyone on it. One raw camera stream is already twice that. Now two hundred robots:

$$200 \times 221 \text{ Mbit/s} = 44,000 \text{ Mbit/s} = 44 \text{ Gbit/s}$$

That is not a Wi-Fi problem, that is a "buy a data centre" problem. And that is one camera per robot, no lidar.

Compress the video (the way a video call does) and each stream drops to about 2 Mbit/s. Two hundred robots then need 400 Mbit/s. Still four access points completely saturated, just for video nobody is watching.

Now send a summary instead. Each robot sends, once a second: its position (x, y, heading), a timestamp, battery percentage, and a state code ("carrying tote", "charging", "stuck"). With message overhead, call it 200 bytes.

$$200 \text{ robots} \times 200 \text{ bytes} \times 1 \text{ per second} = 40 \text{ kB/s} = 320 \text{ kbit/s}$$

That is 0.3 percent of one access point. The ratio between one robot's raw camera and its summary is about 138,000 to one. This is why the fleet server never sees images in normal operation. It sees positions and states. The robot does perception; it sends conclusions.

The exception is when a person needs to see. In 4.7 an operator will ask for one robot's video, and one compressed stream at 2 Mbit/s is fine. The design rule: **video on demand, for one robot at a time; summaries always, for all of them.**

Message queues: the MQTT idea

With two hundred robots sending and one server receiving, you do not want two hundred direct cables into one box. You put a **broker** in the middle. A broker is a program that receives messages on named channels, called **topics**, and hands them to whoever has asked for that topic. The robot **publishes** to a topic; the server **subscribes** to it.

```
Robot 17 --"fleet/17/status" 1 Hz--> +-----+ --all "fleet+/status"--> Fleet server
Robot 18 --"fleet/18/status" 1 Hz--> | Broker | --"fleet/17/status"----> Operator screen
Fleet srv --"fleet/17/task" on event-> | (MQTT) | --"fleet/17/task"-----> Robot 17
                                     +-----+
```

This is the same publish and subscribe idea as a ROS 2 topic (level 2), and it exists with or without ROS. The most common broker protocol for robots talking to a server over unreliable links is **MQTT** (an old name, originally "Message Queuing Telemetry Transport"; today just treat it as a name). MQTT is popular for three reasons that map directly onto the problems above:

1. Messages are tiny, a few bytes of overhead, so summaries stay small.
2. Each message can ask for a delivery level: "fire and forget" for status (a lost one is replaced a second later) or "deliver at least once" for a task (must not be lost).
3. The broker can hold a **last will**: a message it sends on the robot's behalf if the robot goes silent. That is how the server learns robot 17 dropped off the network without robot 17 saying anything.

The trade-off: a broker is one more box that can fail. You accept it because decoupling two hundred senders from the server matters more than one extra box, and you run two brokers so one can die.

Store-and-forward when offline

A robot behind a metal rack loses Wi-Fi for ten minutes. What happens to the data it wanted to send?

Store-and-forward means: write it to local disk, keep working, send it when the link comes back. At 1 kB/s of logs and status, ten minutes is 600 kB. A day is 86 MB. A cheap flash drive holds weeks. So the robot never throws data away just because the network is down.

The design decision inside store-and-forward is *what to send first when the link returns*. The current status, so the server knows the robot is alive now. Then anything safety related (a near-miss). Then the backlog of routine logs, slowly, so the catch-up does not starve the robots that are online. That is a priority queue on the robot, and [tutorial/05-heaps-trees-design.md](#) is where you build one.

Meanwhile, what does the robot *do* while offline? It finishes its current task, which it already has. If it finishes and has no new task, it waits or goes to a safe spot. It never guesses. The rule from 4.2 again: offline must be safe, and "waiting" is safe.

4.4 Fleet-level planning

Traffic: a path is a claim on space and time

One robot plans a path as a list of cells. Two robots must plan paths that do not put both of them in the same cell at the same time. That adds time as a dimension. A path becomes a list of (cell, time step).

Here is a five by five patch of warehouse. Robot X wants to cross left to right along row 2. Robot Y wants to cross top to bottom along column 2.

```
      col 0  1  2  3  4
      +-----+
row 0 |  |  | Y |  |  |   Y: starts (2,0), goes down column 2
      +-----+
      |  |  |  |  |  |
      +-----+
```

```

row 1 | | | . | | |
      +---+---+---+---+
row 2 | X | . | ? | . | . |      X: starts (0,2), goes right along row 2
      +---+---+---+---+
row 3 | | | . | | |
      +---+---+---+---+
row 4 | | | . | | |
      +---+---+---+---+

```

Both move one cell per time step. X is at column t at time t . Y is at row t at time t . At $t = 2$, X is at (2,2) and Y is at (2,2). Collision. A single-robot planner never sees this, because each robot's map shows the cell as empty.

Three simple ways to solve it. Real warehouses use a mix.

1. Reserve cells with time. The fleet server keeps a **reservation table**: a set of (cell, time) pairs that are taken. X plans first and reserves (0,2) at $t=0$, (1,2) at $t=1$, (2,2) at $t=2$, and so on. Y plans second, treating any reserved (cell, time) as a wall. Y finds that (2,2) at $t=2$ is taken, so it waits one step at the start: (2,0) at $t=0$ and $t=1$, then (2,1) at $t=2$, (2,2) at $t=3$. No conflict, one step lost. Checked: with Y delayed by one step, no cell is shared at any time, and there is no "swap" (X and Y passing through each other).

This is *A on a grid with a third coordinate, time*. The A you build in Part C of tutorial/03-grids-bfs-dfs.md is exactly the planner box here; the only change is that the "is this cell free" check also asks "at this time step". The cost is that the reservation table grows with robots times path length, and a robot that is late breaks everyone's reservations behind it. You accept it because in a warehouse the floor is yours, the robots are punctual, and near-perfect use of narrow aisles matters more than simplicity.

2. Priority order. Number the robots. When two conflict, the lower number goes and the higher number waits. No table needed; each robot only needs to know the plans of robots with lower numbers. Simple and robust. The cost is that robot 200 can be starved forever in a busy crossing. You accept it in small fleets where simplicity matters more than fairness. Priority can be smarter than a fixed number: "robot carrying a tote beats an empty one", "robot with lower battery beats one with more".

3. One-way lanes. Redraw the map so that aisles are one-way, like a road system. Then head-on conflicts cannot happen at all, and only crossings need rules. This is the cheapest solution because it removes the problem instead of solving it. The cost is longer paths: a robot may drive round three sides of a block to reach a shelf that was two metres behind it. You accept it when the fleet is large and the aisles are narrow, because never deadlocking matters more than the shortest route.

One-way layout of the same patch:

```

          col 0  1  2  3  4
      +---+---+---+---+
row 0 | > | > | > | > | v |    > east only
      +---+---+---+---+
row 1 | ^ |   |   |   | v |    v south only
      +---+---+---+---+
row 2 | ^ |   |   |   | v |    ^ north only
      +---+---+---+---+
row 3 | ^ |   |   |   | v |    < west only
      +---+---+---+---+
row 4 | ^ | < | < | < | < |
      +---+---+---+---+

```

Under any of these, the global route is a **graph** problem: junctions are nodes, lane segments are edges, and the edge weight is travel time, which the server can raise for a congested lane. That is Dijkstra from tutorial/04-graphs-dijkstra.md, run on the server, with weights that change every few seconds as traffic changes.

The split between server and robot on planning is then:

Level	Where	Input	Output	How often
Route (which aisles)	Server	fleet positions, one-way map, congestion	list of junctions	on new task, and re-plan if a lane blocks
Path (which cells, when)	Server or robot	route, reservation table	list of (cell, time)	on new route
Local (next 2 m)	Robot only	lidar, path	speed and steer	20 Hz

Task assignment: which robot takes which order

Orders arrive. Robots are free. Match them. Two common ways:

Nearest-first. Take orders in the order they arrive. For each one, give it to the nearest free robot. Simple, fast, easy to explain.

Auction. When an order appears, announce it to all robots. Each robot replies with a **bid**: its cost to do this order (distance, or time, including any work it already has). The lowest bid wins. Robots that are far away or busy bid high and lose. This is the same matching problem, but the robots do the costing, so it scales and keeps working if the server's picture of a robot is slightly out of date.

Both are **greedy**: they make the best choice for one order at a time and never go back. A worked example on a straight aisle shows why that can hurt.

Position: 0 1 2 3 4 5 6 7 8 9 10 11 12 13
 R1 A R2 B R3 C

R1 at 0, R2 at 5, R3 at 12. Orders arrive in the order A (4), B (6), C (13).
 Cost = distance along the aisle.

Nearest-first, orders in arrival order:

Order	Free robots and distances	Chosen	Cost
A at 4	R1: 4, R2: 1, R3: 8	R2	1
B at 6	R1: 6, R3: 6	R1	6
C at 13	R3: 1	R3	1
		Total	8

Now try all six ways of pairing three robots with three orders and add up the distances. The best is R1 to A (4), R2 to B (1), R3 to C (1): **total 6**. Nearest-first gave 8. It grabbed R2 for A because R2 was closest, and left R1 with a long walk to B. Greedy is a third worse here, and in a real warehouse the gap can be bigger.

The trade-off: the best matching of N robots to N orders can be found exactly (the method is called the Hungarian algorithm; you do not need its details), but it costs roughly N cubed work and needs all orders known up front. Nearest-first costs almost nothing and works one order at a time as they arrive. Most fleets run nearest-first or an auction because orders arrive continuously and a decision in ten milliseconds matters more than a perfect answer. Then they add a fix: every few seconds, re-run assignment on all *unstarted* orders and swap if it helps. That recovers most of the gap.

Note that a plain "lowest bid wins, one order at a time" auction gives the same total of 8 on this example, because it is the same greedy choice made by the robots instead of the server. The auction's advantage is not a better answer; it is that the robots compute their own costs, so a robot with a low battery or a half-finished job can bid high on its own without the server needing to model it.

4.5 Maps and data

Who owns the map

With one robot, the map is a file on the robot. With two hundred, the fleet server owns the **master map** and each robot holds a copy. The map is versioned: every change gets a new number, and every robot reports which version it is running.

```

Robot 41  --"new obstacle at (12,7), map v83"---> +-----+
                                           | Fleet server | --"map v84" once--> Robot 1
                                           | master map   | --"map v84" once--> Robot 2
                                           | v83 -> v84   | ...
Operator  --"confirm obstacle"-----> +-----+ --"map v84" once--> Robot 200
                                           on event

```

Robots do not edit the master map directly. They *propose* changes ("I saw a pallet here that is not on the map") and the server merges proposals. A single robot's false detection must not delete a corridor for two hundred robots. In practice: a change seen by one robot is marked "provisional" and routes avoid it; a change seen by three robots, or confirmed by the operator, becomes permanent.

How updates get to robots: the server publishes "map v84 available". Each robot fetches the difference from its version (only the changed cells, a few kilobytes) rather than the whole map (megabytes). It swaps to the new version at a safe moment, between tasks, never in the middle of a narrow aisle. A robot on an old map version is allowed to keep working, but the server plans its routes on the *older* of the two versions so both agree on what exists.

Versioning is the whole trick. Without a version number you cannot tell whether robot 90's "there is no obstacle at (12,7)" is a disagreement or just old news.

What data to collect from the fleet, and why

You have two hundred robots driving eight hours a day. That is 1,600 robot-hours a day of experience. It is the most valuable thing the fleet produces after the orders themselves, because it tells you where the design is wrong. Collect, at a minimum:

Data	Why	Size
Failures: what stopped, where, when, last 30 s of state	Find the top three causes and fix them.	small; a few kB per event
Near-misses: emergency stops that triggered, obstacles closer than the safety margin	Failures you got away with. Same causes, ten times more samples.	a few kB per event, plus a short video clip on request
Battery curves: charge level against time, for each robot	Spot a robot whose cell is dying before it strands a tote in an aisle. Plan charger count.	1 byte per second per robot
Task times: assigned, started, finished	Measure whether assignment changes help. Feed the analytics.	tiny
Map proposals accepted and rejected	Tune the perception threshold.	tiny

Notice what is not on the list: continuous video. It is too big (4.3) and mostly useless. You keep a rolling buffer of the last 30 seconds on the robot and upload it only when an event happens. That is store-and-forward with a trigger.

Privacy of camera data on pavements

A warehouse camera sees shelves. A pavement delivery robot's camera sees people's faces, car number plates, and the inside of front windows. In the UK that is personal data, and the rules (the UK General Data Protection Regulation, GDPR) apply. You do not need legal detail, but you need design rules:

- Process on the robot, send conclusions. "Person 2 m ahead" leaves the robot. The image does not. This is the same rule as bandwidth, for a different reason.
- If an image must leave the robot (a crash, a remote operator), blur faces and plates on the robot before sending, and delete the clip after a fixed time unless it is part of an incident.
- Never keep video "just in case". Storage you do not have cannot leak.

The trade-off: blurring costs compute on the robot and makes the operator's picture worse. You accept it because not becoming a walking surveillance camera matters more than a crisp teleoperation view.

4.6 Updates and configuration

Over-the-air updates

An **over-the-air (OTA) update** is new software sent to the robot over the network and installed without anyone touching the robot. It is the only practical way to update two hundred robots, and it is also the fastest way to break all two hundred at once.

A full software image for a robot computer is around 500 MB. At 20 Mbit/s per robot that is about three and a half minutes each. Two hundred robots is 100 GB; on one 100 Mbit/s access point, over two hours if nothing else used the network. So updates download in the background, slowly, while robots work, and only *install* when the robot is docked. Downloading and installing are separate steps with separate commands.

The robot always keeps two copies of its software: the one running and the one being installed. The install writes the new copy, checks it, then flips a switch to boot from it. If the new copy fails to start, the switch flips back. This is called an A/B partition scheme and it is what makes the next two ideas possible.

Staged rollout: 1, then 10, then all

Never send an update to the whole fleet at once. Stage it:

Stage 1:	1 robot	soak 1 day	Does it boot? Does it drive? Any new errors?
	v (only if stage 1 is clean)		
Stage 2:	10 robots	soak 1 day	Same questions, across different aisles and shifts.
	v (only if stage 2 is clean)		
Stage 3:	all 200	over a shift	Rolling: never more than 10 percent installing at once.

"Clean" is defined before you start: no new failure types, task time not worse than 2 percent, no near-miss increase. If any number is bad, you stop and roll back the robots that have it.

The trade-off: a staged rollout takes three days instead of an hour. You accept it because one broken robot for a day costs far less than two hundred broken robots for an hour, and because you can never test a warehouse floor as well as the warehouse floor tests itself.

Rollback

Rollback means putting the previous version back. With A/B partitions it is one command: flip the switch, reboot. Two rules:

- Every update must be rollback-safe. If version 84 changes the format of the map file, version 83 must still be able to read what 84 wrote, or the rollback breaks worse than the update did.
- Rollback is automatic when the robot itself detects it is unhealthy after an update (fails to start, cannot reach the broker, watchdog fires). It does not wait for a person.

Config versus code

Code is the program. **Configuration** (config) is the numbers the program reads at startup: maximum speed, safety margin, which map, which broker address. Keep them separate.

Changing config is cheap: a small file, no reboot, applied at the next safe moment. Changing code is expensive: staged rollout, days. So design so that anything an operator might need to change on a Tuesday afternoon ("slow the robots in aisle F, there is a spill") is config, not code. And version config too: "robot 17 runs code v84 with config v212" is a complete description; "robot 17 has the new settings" is not.

The trade-off: too much in config, and nobody knows what the robot actually does without reading a hundred numbers. A good middle: a handful of operator-facing knobs in config, the rest in code with tests.

4.7 Monitoring and operations

What an operator sees

The operator is the person on the human end of the arrow in 4.1. Their screen is the fleet server's view drawn for a human. A useful one shows, top to bottom:

```
+-----+
| FLEET: 200 robots   Working 171   Charging 21   Idle 5   NEEDS HELP 3   |
+-----+
| ALERTS (newest first)                                     |
| !! 14:02 Robot 17   stuck 90 s, aisle B row 12   [view] [assist] |
| !  13:58 Robot 103  battery 9 %, 400 m from charger |
| !  13:51 Robot 66   map proposal: obstacle at (12,7), unconfirmed [ok] |
+-----+
| MAP                                                         | FLEET HEALTH (last hour) |
| . . . . . | Orders/hour      480   (target 500) |
| . 3 . . . . 9 . . . . 41 . . | Avg task time    3.1 min |
| . . . . 17! . . . . . | E-stops          2      |
| . . 88 . . . 12 . . . 55 . . | Wi-Fi dropouts   14     |
+-----+
```

The design rule for a dashboard is the same as for the network: **exceptions, not everything**. The operator cannot watch two hundred dots. They watch three alerts and a handful of numbers that drift when something is wrong across the fleet (orders per hour falling, dropouts rising).

Alerts

An alert is a rule on the server that turns data into a line on that screen. Three rules make alerts useful rather than noise:

1. **Every alert says what to do.** "Robot 17 stuck" is a fact. "Robot 17 stuck 90 s, aisle B row 12, [assist]" is an alert.

2. **Alerts have levels, and the levels mean something.** Red: a person acts now. Amber: a person acts this shift. Grey: logged, look at it weekly. If everything is red, nothing is.
3. **An alert clears itself.** When robot 17 moves, the line goes. The operator never tidies.

The trade-off: a threshold too low floods the screen; too high and a real stuck robot waits ten minutes. You tune with data from 4.5, and you start with thresholds that fire rarely, because an operator who trusts the screen matters more than catching every hiccup.

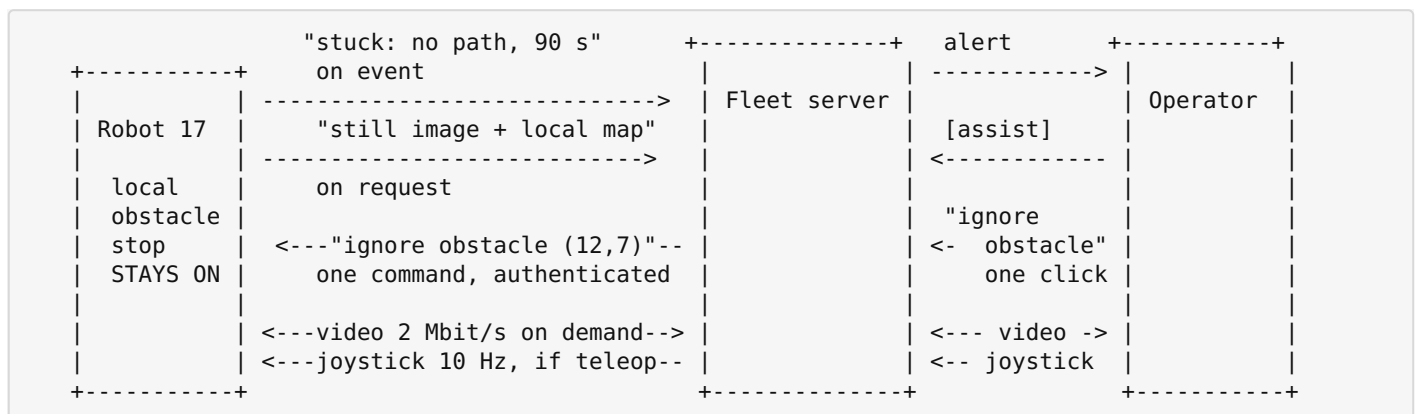
Remote assistance and teleoperation

A robot is stuck. Its planner has no path: a pallet blocks the aisle, or a person parked a trolley across it, or it thinks there is an obstacle that is not there. Options in order of cost:

1. **Remote nudge.** The operator looks at the robot's map view and a still image, and clicks "ignore that obstacle" or "reverse 1 m and re-plan". The robot does the driving. One picture, a few bytes back. This solves most cases.
2. **Teleoperation.** The operator drives the robot with a joystick over a live video stream. This is the expensive case: one 2 Mbit/s stream and a person's full attention.
3. **Send a person.** Someone walks to aisle B.

Teleoperation has a latency limit. Video from the robot's camera to the operator's screen (called glass-to-glass) over 4G is around 300 milliseconds. At 1.5 m/s, the robot moves 0.45 m before the operator sees anything. At 0.5 m/s it is 0.15 m. So teleoperation is done at a crawl, and the robot's own obstacle stop stays on. The operator is steering, not overriding safety. Rule from 4.2, again.

The human-in-the-loop path, drawn



Every command on the path is logged with who sent it and when. Every command goes through the fleet server, never operator to robot directly, so there is one place to check permissions (4.9) and one log.

Expected load: if each robot needs help twice per eight-hour shift, for three minutes each, that is 1.25 percent of the time. For fifty robots, on average less than one is stuck at a time. One operator per fifty robots is a common starting ratio and this arithmetic is why.

4.8 Safety cases and standards, in plain words

What a safety case is

A **safety case** is a written argument that the robot is safe enough to use, backed by evidence. It has three parts: what could go wrong, what you did about each thing, and how you know it worked. It is a document, not a test. Regulators, insurers, and a warehouse owner's safety officer read it. You write it during design, not after, because the cheapest fix for a hazard is a design that cannot produce it.

The main computer sends a heartbeat. If it stops, the safety controller cuts power. That is the **watchdog** from level 3, in hardware. The main computer can *request* a stop; it can never *prevent* one. And the fleet server cannot reach the safety controller at all, which is the point of 4.2's rule made physical.

"Safety-rated" means the part has been tested and certified to a standard to fail in a known, safe way, and its failure rate has been measured. It costs more. You accept it because a certified stop that works when the main computer is on fire matters more than the price of a second small board.

Three standards by name

You will see these names in job adverts and design reviews. One line each; no legal detail.

- **ISO 13482**: the safety standard for personal care robots, meaning robots that work close to ordinary people, such as a delivery or assistance robot on a pavement or in a home.
- **ISO 3691-4**: the safety standard for driverless industrial trucks, which covers warehouse robots and automated guided vehicles moving goods around a site.
- **ISO 26262**: the functional safety standard for road vehicles' electronics, which is what autonomous car software and hardware is assessed against.

A standard tells you what evidence a safety case needs. It does not design the robot for you.

4.9 Security basics

Safety is about accidents. **Security** is about people who want the robot to do the wrong thing on purpose. Three basics for a fleet.

Signed updates. Every software image is signed with a private key that only your release process has. The robot checks the signature with the matching public key before installing. An image without a valid signature is refused, even if it came from the right server, because "the right server" is a network address, and network addresses can be faked.

Authenticated commands. Every command that reaches a robot, "go to C7", "ignore obstacle", "install v84", carries proof of who sent it, and the robot checks that this sender is allowed to send this kind of command. An operator can nudge a stuck robot; only the release process can install software; nobody can turn the safety controller off, because there is no command for it. The broker in 4.3 enforces this too: a robot may publish only to its own status topic and subscribe only to its own task topic. Robot 17 cannot send a task to robot 18.

Why an open emergency stop API is dangerous. It seems harmless, even good, to have a network command that stops every robot. Think about who else can send it. A stop command that anyone on the Wi-Fi can send is a button that halts your warehouse. Two hundred robots stopping at once in narrow aisles, some mid-turn, some on ramps, with totes in the air, is itself a hazard, and it is also a way for one person with a laptop to shut a business down. So:

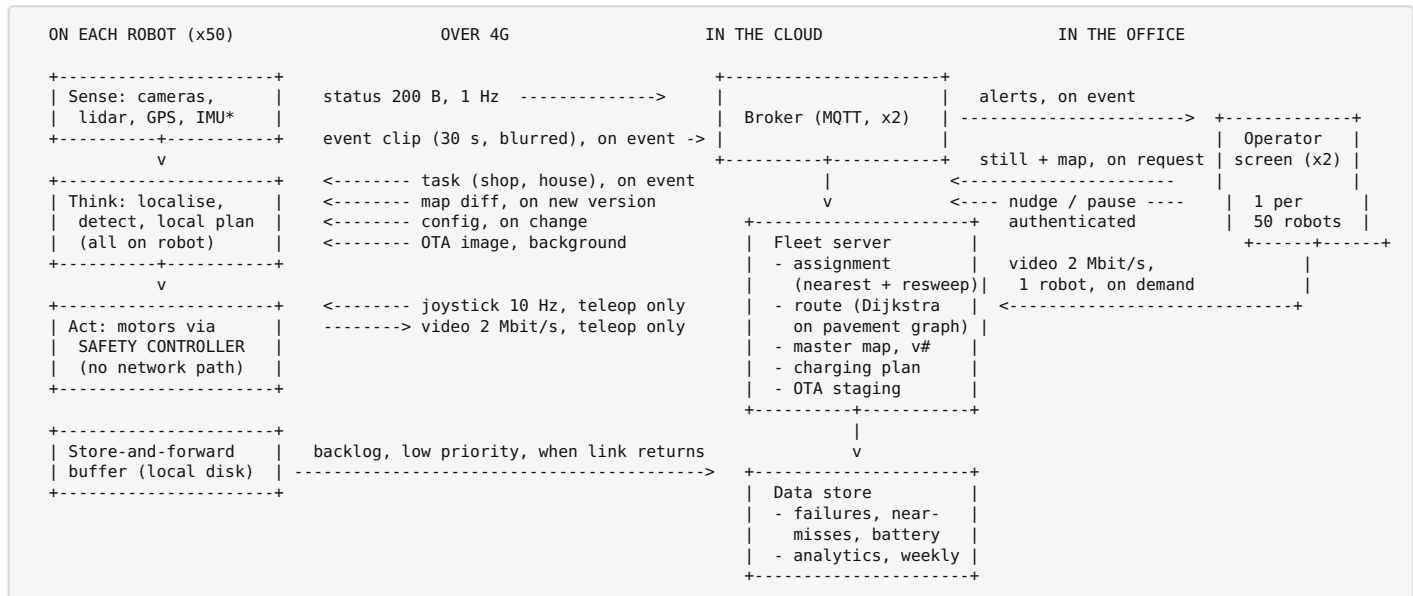
- The physical e-stop is a button, wired to the safety controller, not on the network at all.
- A fleet-wide "pause" command exists, authenticated, limited to two named operator accounts, logged, and it makes robots *finish their current motion and stop in a safe spot*, not cut power.
- Robots reject a pause from anyone else, and report the attempt as an alert.

The trade-off: authentication adds a little latency and a lot of key management. You accept it because a fleet that only its owners can command matters more than a slightly simpler message format.

4.10 Worked example: a 50-robot city delivery fleet

Level 1 asked you to draw a pavement delivery robot. Now scale it. The job: fifty robots delivering from twenty shops to homes across a UK town, on pavements, over 4G, with two operators in an office. Let us design the fleet layer from the ideas above.

The full picture



(*IMU: inertial measurement unit, a chip that senses acceleration and turning rate.)

The numbers behind it, checked:

- Status: 50 robots at 200 bytes, once a second, is 80 kbit/s for the whole fleet. Trivial.
- Per robot on 4G: 1 kB/s of status and logs continuously is 86 MB a day, about 2.6 GB a month. Fits a cheap data plan. One compressed camera stream at 1 Mbit/s continuously would be 10.8 GB a day, 324 GB a month, per robot. That is the whole argument for event clips.
- Teleoperation: if all fifty streamed at once it would be 100 Mbit/s, but they never do. At two stuck events per shift of three minutes each, under one robot on average needs help at any moment. Two operators is ample; one is tight when two robots get stuck together.
- Charging: eight hours running, one charging means about one in nine charging at once, roughly six of fifty. Put eight charging bays in the depot and stagger the schedule so the lunchtime peak has every robot out.
- Teleop latency: 300 ms glass-to-glass, so teleop is capped at 0.5 m/s (the robot moves 0.15 m before the operator sees it), and the safety lidar stop stays active.

Interface table

Arrow	What travels	How often	If it stops
Robot → Broker: status	position, heading, battery, state code, timestamp, map version, code version (about 200 B)	1 Hz	broker's last-will marks robot "silent"; alert after 30 s
Robot → Broker: event clip	30 s of blurred video plus state log, on a failure or near-miss	on event	buffered on robot, sent later
Robot → Data store: backlog	logs since last contact	when link returns, low priority	stays on disk; weeks of space

Arrow	What travels	How often	If it stops
Server → Robot: task	shop id, house address, route as list of junctions, deadline	on assignment, at least once delivery	robot finishes current task, then waits at a safe spot
Server → Robot: map diff	changed cells, from robot's version to newest	on new version	robot keeps old version; server routes it on the old version
Server → Robot: config	speed cap, safety margin, zones	on change	robot keeps last config
Server → Robot: OTA image	signed image, background download	per staged rollout	download resumes; install only when docked
Operator → Server → Robot: nudge	"ignore obstacle", "reverse and re-plan", "pause"; authenticated, logged	on click	robot stays stopped; still safe
Operator ↔ Robot: teleop	video 2 Mbit/s down, joystick 10 Hz up	during assist only	robot stops within 200 ms of last joystick message (watchdog)
Sensors → Safety controller → Motors	obstacle-in-zone, bumper, e-stop, heartbeat	continuous	power cut; nothing on the network can prevent this

Three trade-offs

- 1. Assignment is nearest-first with a re-sweep every ten seconds, not an exact matching.**
Because orders arrive all day and a robot must be moving within a second of an order appearing, responsiveness matters more than the last five percent of distance saved. The re-sweep on unstarted orders recovers most of the gap, as in the 8 versus 6 example.
- 2. Perception and every safety decision stay on the robot; the cloud only sees conclusions.**
Because a 4G dropout under a railway bridge must leave the robot safe and still moving, and because faces on pavements must not leave the robot. This costs a more powerful computer on every robot, fifty times over. Safe-when-offline and privacy matter more than a cheaper robot.
- 3. The fleet-wide pause makes robots finish their motion and pull in, rather than cutting power, and it is limited to two named accounts.** Because fifty robots stopping dead across a town's pavements is itself a hazard, and a remotely triggerable hard stop is an attack surface. Controlled stopping and a small, logged set of senders matter more than the instinct that "stop everything now" should be one click for anyone.

4.11 Try it

Do these on paper, with boxes, labelled arrows and numbers.

- 1. Warehouse to 1,000 robots.** Redo the bandwidth arithmetic in 4.3 for a thousand robots. Does the summary approach still fit on the network? What about event clips if each robot has one event an hour? Where is the first thing that breaks?
- 2. Drone inspection fleet.** Ten drones inspect wind turbines on a hill with no 4G, only a base station van with Wi-Fi at the bottom. Draw robot, fleet server, operator. Which boxes move into the van? What does store-and-forward look like when a drone is out of range for twenty minutes and its battery lasts twenty-five?

3. **Traffic on your own grid.** Draw a 6 by 6 grid with two aisles crossing. Put three robots on it with start and goal cells. Build the reservation table by hand for priority order 1, 2, 3. Then redraw the grid with one-way lanes and count the extra cells each robot drives. Which do you choose, and why, in one level-1 sentence?
 4. **Staged rollout gone wrong.** Version 85 reaches stage 2 (10 robots). Task time is fine. Near-misses go from 2 per hour to 5 per hour across those ten. Write down, in order, what the server does automatically, what the operator sees, and what a human must decide.
 5. **Hazard row.** Add two rows to the hazard table in 4.8 for the warehouse fleet, not the pavement robot: one hazard involving a person on foot in an aisle, one involving two robots. Fill in every column, including "Left over".
-

4.12 In your own words

A fleet is many complete robots plus one new system around them: a server that assigns tasks, routes traffic, owns the map and stages updates, and a person who handles exceptions. The network between them is slow, shared and often absent, so robots send small summaries, keep everything needed for a safe stop on board, and buffer what they cannot send. Traffic is planning with time added; assignment is matching that is usually greedy for speed. Updates go out in stages with a rollback ready. A safety-rated stop lives on its own circuit, and no network command can reach it. Everything else is the same four words as level 1: components, interfaces, data flow, trade-offs, just with more boxes.

4.13 Check yourself

Answer each in two or three sentences, without looking back. If one is hard, reread that section.

1. Name the five things that change when you go from one robot to two hundred, and say which box handles each.
2. What is the one rule that decides whether a job runs on the robot or in the cloud? Give one job on each side and the reason.
3. One raw camera is about 221 Mbit/s and a summary is about 200 bytes a second. Roughly what is the ratio, and what does that imply for what the fleet server sees?
4. What is a broker, what is a topic, and what is a "last will" in MQTT for?
5. A robot loses Wi-Fi for ten minutes. What does it do with its data, what does it do with its task, and what does it send first when the link returns?
6. Explain the reservation table in one paragraph. What changes about A* to use it?
7. In the aisle example, why did nearest-first cost 8 when the best was 6? What cheap fix recovers most of the gap?
8. Why does the map have a version number, and what does the server do when a robot is on an older version than the master?
9. Describe a staged rollout, what "clean" means at each stage, and why A/B partitions make rollback safe.
10. Why must the stop circuit be separate from the main computer, and why is an open network emergency stop dangerous even though it sounds safe?

Level 5. Design questions, worked end to end

This is the top level. Levels 1 to 4 gave you the words (component, interface, data flow, trade-off), the common robot boxes, the three questions to ask about any arrow (latency, throughput, failure), and how to make a design safe. This level puts all of it to work on six real robots, the way you would if someone handed you a blank whiteboard and forty minutes.

It is also the only level that talks about hiring. A **design question** is a conversation in which someone gives you a vague request ("design a robot that does X") and watches how you turn it into boxes, arrows, numbers and trade-offs. Nobody expects a finished design. They want to see a method. This level teaches the method, then applies it six times so you can see what "good" looks like at the end of each step. Read it as coaching, not as lines to recite.

Rough time: two evenings for the method and the first three designs, one more for the rest.

5.1 The method

Forty minutes is short. The method below is seven steps with a time for each. The times add up to forty. Most people who struggle with design questions fail step 1 or step 2 and then run out of time drawing boxes for the wrong robot.

Step 1	Step 2	Step 3	Step 4	Step 5	Step 6	Step 7
Ask what it must do and never do	Numbers first	Draw sense -> think -> act	Label the arrows	Go deep on the 3 hardest arrows	Name the trade-offs	Say what to build first
~5 min	~5 min	~5 min	~5 min	~12 min	~5 min	~3 min

Step 1. Ask what the robot must do and must never do (5 minutes). The request is vague on purpose. Before you draw anything, narrow it with five questions. Write the answers in a corner of the board and point back at them all the way through.

1. **Who uses it and where?** A warehouse with trained staff is not a pavement with children.
2. **What does "done" look like?** Picks per hour, deliveries per day, blades per battery. One number you can be measured against.
3. **What must it never do?** Hit a person, drop a parcel in the road, fly into the blade, move when the surgeon has not asked it to. This is the safety list. Every design later gets checked against it.
4. **What is fixed and what is free?** Is the hardware chosen already? Is there a budget? Is there a deadline? Is there a network?
5. **What happens today without the robot?** The manual process tells you the real requirements. A human picker in a bin takes a few seconds per item; that is your target.

If the person asking says "you decide", decide, say the assumption out loud once, and move on. Do not spend ten minutes on questions.

Step 2. Numbers first (5 minutes). Before boxes, write down five numbers. Speed of the robot. Size of the space it works in. The rate of the main sensor. The battery and how long a shift is. The budget or the number of units. Each number will decide something later: speed decides stopping distance, which decides sensor range, which decides sensor cost. Rough is fine. "About 1 metre per second" is a number. "Fast" is not.

Step 3. Draw sense, think, act (5 minutes). Left to right, as in Level 1. Start with three boxes and split each one only when you have a reason. Six to ten boxes is the right size. Group anything bigger into a box with a name and say you will open it if asked.

Step 4. Label the arrows (5 minutes). For every arrow: what travels and how often. An image at 10 Hz. A pose at 50 Hz. A goal on change. This is where you catch the arrow that carries 500 megabytes a second over Wi-Fi, before you build it.

Step 5. Pick the three hardest arrows and go deep (12 minutes). Not all arrows are equal. Find the three that decide whether the robot works: usually one about latency (the loop that stops the robot), one about failure (what stops working when the network drops), and one about scale (what breaks when there are two hundred robots instead of one). For each, say the number, say what happens when it is missed, and say the fix.

Step 6. Name the trade-offs (5 minutes). For each big choice say "X because Y matters more than Z here". Central planner because global optimality matters more than the single point of failure, which we cover with a hot standby. Cheaper lidar because the robot walks at 1.5 m/s and never needs to see past 10 metres. Three or four of these is plenty.

Step 7. Say what you would build first (3 minutes). A design nobody can start on is not a design. Name the thinnest version that proves the riskiest part: one robot, one route, a remote stop button. Then say what you would add second and third.

The one-page checklist

- ☐ 5 questions answered, written in the corner (who, done, never, fixed, today)
- ☐ 5 numbers written down (speed, space, sensor rate, battery, budget)
- ☐ sense -> think -> act, 6 to 10 boxes, left to right
- ☐ every arrow labelled: what + how often
- ☐ 3 hardest arrows chosen: one latency, one failure, one scale
- ☐ for each hard arrow: the number, what happens if missed, the fix
- ☐ 3 or 4 trade-offs said as "X because Y matters more than Z"
- ☐ safety is on the robot, not in the cloud
- ☐ what to build first, second, third
- ☐ checked the design against the "must never" list

Each of the six designs that follow uses exactly these steps as headings, so you can see the method applied rather than described.

5.2 A robot arm picking items from a bin with one camera

This is exercise 1 from Level 1. The request: "Design a robot arm that picks items out of a bin and puts them in a box. There is one camera above the bin."

Step 1. What it must do and never do

Who and where: a warehouse packing station. Trained staff walk past, sometimes reach into the bin to fix a jam. Done means about 600 picks an hour with less than one jam an hour that needs a person. Never: touch a person, crush an item, drop an item outside the box. Fixed: a six-joint arm with a suction gripper and one colour-plus-depth camera (a camera that gives a colour image and, for every pixel, a distance) are already bought. Today: a human picks at roughly one item every four seconds, so 600 an hour (one every six seconds) is acceptable.

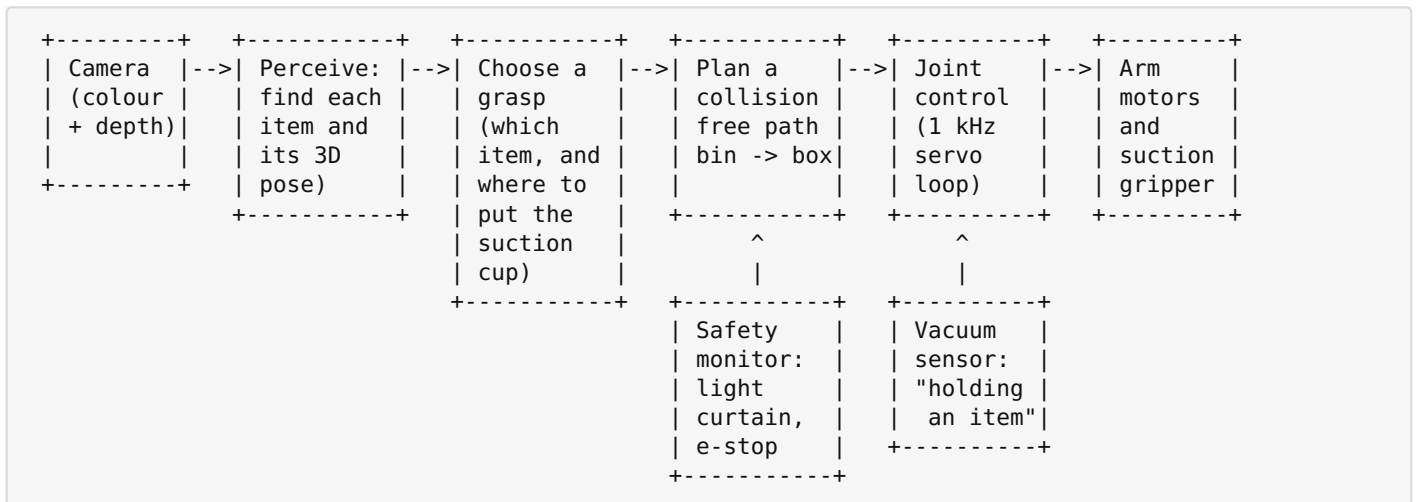
Step 2. Numbers first

Number	Value	Why it matters
Arm tool speed	up to 1 m/s	sets stopping distance and the safety zone

Number	Value	Why it matters
Bin to box distance	0.8 m	sets the travel time per pick
Camera	1280 by 720 colour plus depth, taken once per pick	4.6 MB per capture
Pick target	600 per hour, one every 6 s	the budget every step must fit inside
Joint control loop	1000 times a second (1 kHz)	the arm's inner loop, 1 ms per step

One capture is 1280 by 720 pixels times 3 bytes for colour (2.8 MB) plus 2 bytes per pixel for depth (1.8 MB). If you streamed it at 10 Hz that would be 46 MB/s. You do not need that. One capture per pick, six a minute, is almost nothing.

Step 3. Draw sense, think, act



The **light curtain** is a row of light beams at the edge of the cell; a hand breaking a beam is a signal. The **safety monitor** is a separate small box whose only job is to stop the arm. It is not part of the perception computer, on purpose (Level 4).

Step 4. Label the arrows

Arrow	What travels	How often
Camera → Perceive	colour and depth image, 4.6 MB	once per pick, about every 6 s
Perceive → Choose grasp	list of items: 3D position, orientation, a "how sure" score	once per pick
Choose grasp → Plan path	one grasp pose (x, y, z, angle) and one drop pose	once per pick
Plan path → Joint control	a path: list of joint angles with times, about 200 points	once per pick
Joint control → Motors	a torque or velocity per joint	1 kHz
Motors → Joint control	actual joint angle and current per joint	1 kHz
Vacuum sensor → Choose grasp	"item held" or "lost it"	100 Hz
Light curtain → Safety monitor	"beam broken"	immediately, wired, under 10 ms

Arrow	What travels	How often
Safety monitor → Joint control	"stop now"	immediately, a physical line, not a message

Step 5. The three hardest arrows

Latency: the light curtain to a stopped arm. The tool moves at up to 1 m/s. From a hand breaking the beam to the arm being still: detection 50 ms, controller reaction 100 ms, braking 200 ms, total 0.35 s. In that time the arm can travel 0.35 m. A person's hand moving in at 1.6 m/s (the standard figure used for safety distance) covers 0.56 m. So the light curtain must sit at least $0.35 + 0.56$, about 0.9 m, outside the furthest point the arm can reach. Fix if there is not that much room: slow the arm when a person is near (a zone sensor), which shrinks the 0.35 m.

Failure: the perception box returns rubbish. A shiny bag gives the depth camera no reading. A transparent item is invisible. The grasp chooser must have a "how sure" score on every item and refuse to pick under a threshold. After two refusals on the whole bin, it shakes the bin (a small tilt) and retries once, then calls a person. With 98% grasp success at 600 picks an hour you still get 12 failures an hour, so the recovery path is not an edge case, it runs every five minutes.

Scale: the pick cycle budget. Capture 0.1 s, perceive 0.3 s, choose 0.1 s, plan 0.2 s, move to grasp 1.5 s, grasp 0.3 s, move to box 1.5 s, release 0.2 s, return 1.2 s. That is 5.4 s, which gives 666 picks an hour, just above the target. The two 1.5 s moves dominate. The trick that buys time: take the next picture while the arm is over the box, so capture and perception of pick N+1 overlap with the move of pick N. That takes 0.4 s out of the cycle.

Step 6. Trade-offs

Choice	Chosen because	At the cost of
Suction cup, not fingers	most parcels are flat boxes and bags, and suction is fast and simple	cannot pick mesh bags or heavy items; needs the vacuum sensor
One capture per pick, not a video stream	freshness is not needed between picks, and it keeps the compute small	if a person nudges the bin mid-cycle the picture is stale, so the vacuum sensor catches the miss
Separate safety monitor with wired stop	a person's safety must not depend on the perception computer being healthy	a second box, more wiring, more cost
Slow zone instead of a full cage	staff need to reach in often; a cage would be opened all the time	needs a zone sensor and testing to prove the slow speed is safe

Step 7. What I would build first

One arm, one bin, one item type, a fixed drop point, a full physical fence with a door interlock (the arm cannot move while the door is open). No grasp chooser at all: pick the item whose centre is highest. Measure the real success rate and cycle time. Then add: the "how sure" score and the recovery path, then mixed item types, then the light curtain and slow zone to replace the fence.

Questions someone might push on, with answers

- *Why not put the camera on the arm?* A wrist camera sees around occlusions but you have to move the arm to take a picture, which costs cycle time. One fixed camera above the bin is faster. Add a wrist camera later if the fixed one keeps missing items at the bin's edges.
- *Does this need ROS 2?* No. The boxes and arrows exist without it. ROS 2 gives you a **node** (a running box), a **topic** (a named stream, like the image arrow), a **service** (ask and wait, like "plan a path"), and

an **action** (a long job with progress, like "move to the box"). It would be a natural fit for the perception to control side. The 1 kHz joint loop and the safety monitor would not go through ROS 2 messaging.

- *Where is the collision map for the planner?* The bin walls and the box are fixed and known. Items in the bin come from the depth image. A planner such as the A* in code/03-grids-bfs-dfs/astar works on a grid; an arm planner works in joint space, but the idea of "search from start to goal, avoiding blocked cells" is the same.

5.3 A pavement delivery robot

Exercise 2 from Level 1. The request: "Design a robot that drives on pavements from a shop to a house with a bag of groceries." This one ties Levels 2, 3 and 4 together: the standard boxes of a mobile robot (Level 2), the latency, throughput and failure questions on each arrow (Level 3), and the safety layer that sits outside the clever software (Level 4).

Step 1. What it must do and never do

Who and where: a town centre pavement shared with pedestrians, prams, dogs and bikes. Done: deliver within 30 minutes over routes up to 2 km, 20 deliveries a day per robot, and the customer opens the lid with a code. Never: hit a person, roll into the road, tip over on a kerb, get stuck where nobody can reach it, lose the goods. Fixed: a six-wheeled base with a lockable box, 4G mobile data, a budget that rules out a car-grade lidar. Today: a cyclist does this, and takes about 10 minutes for 2 km.

Step 2. Numbers first

Number	Value	Why it matters
Speed	1.5 m/s (5.4 km/h, brisk walking)	stopping distance, and how far a delayed command lets it travel
Route	2 km, so 22 minutes at 1.5 m/s	fits the 30 minute promise with time for crossings
Battery	500 watt-hours, average draw 150 W	3.3 hours, which is 18 km of driving, so about 4 round trips
Cameras	2 front cameras, 1280 by 720, 15 Hz	83 MB/s raw, must be processed on the robot, never sent raw
Lidar	small 16-beam, 300,000 points a second	4.8 MB/s, sees 2 to 20 m, enough at walking speed
4G link	100 to 300 ms round trip, a few Mbit/s up	rules out remote driving, allows remote supervising

Step 3. Draw sense, think, act

```

+-----+ +-----+ +-----+ +-----+ +-----+ +-----+
| Cameras |->| Perceive: |->| Local  |->| Path  |->| Motion |->| Wheel  |
| Lidar   | | people,  | | map:   | | planner: | | control | | motors |
+-----+ | kerbs,   | | free   | | route + | | (speed, | +-----+
+-----+ | crossings | | space  | | avoid  | | turn)   |
| Wheel   | +-----+ | around me | +-----+
| encoders|->+-----+ +-----+ ^ ^
| IMU     | | Localise: |-----+ |
| GPS     | | where am | +-----+ +-----+ +-----+
+-----+ | I on the  |->| Mission: | | Remote | | Safety |
          | route map | | which   | <-| supervisor| | monitor: |
          +-----+ | delivery, | | (human, | | bumper, |
                   | next stop | | via 4G) | | tilt,   |
                   +-----+ +-----+ | lidar  |
                                           | near   |

```

```
| field |
+-----+
```

IMU is an inertial measurement unit: a chip that measures turn rate and acceleration. **GPS** gives a position from satellites, good to a few metres, worse between buildings. **Localise** combines encoders, IMU, GPS and what the cameras see against a stored route map, to answer "where am I" to about 0.2 m; this is the Kalman filter idea in code/06-robotics/01_kalman_1d.cpp with more inputs. The **route map** is made once by driving the route with a person: a grid of the pavement 2 km long and 20 m wide at 0.1 m cells is 4 million cells, 4 MB at one byte each, small enough to live on the robot.

Step 4. Label the arrows

Arrow	What travels	How often
Cameras → Perceive	two images, 1280 by 720	15 Hz
Lidar → Perceive and Safety monitor	point cloud, 300k points/s	10 Hz sweeps
Encoders, IMU, GPS → Localise	wheel ticks 100 Hz, turn rate 200 Hz, position 10 Hz	as listed
Perceive → Local map	boxes around people and bikes with speed; kerb line	15 Hz
Localise → Path planner	(x, y, heading) on the route map, with a "how sure"	20 Hz
Mission → Path planner	the next waypoint on the route	on change
Path planner → Motion control	target speed and turn rate for the next 2 s	20 Hz
Motion control → Motors	a speed per wheel	50 Hz
Robot → Remote supervisor	compressed video 2 Mbit/s, position, state	video continuous, state 1 Hz
Remote supervisor → Mission	"go", "wait", "take the other path", "stop"	on change, seconds apart
Safety monitor → Motion control	"stop", overriding everything	wired, immediate

Step 5. The three hardest arrows

Latency: seeing a child to being stopped. Perception takes 100 ms, planning 50 ms. In that 0.15 s the robot travels 0.22 m. Braking from 1.5 m/s at 3 m/s squared takes 0.5 s and 0.38 m. Total stopping distance 0.6 m. So anything the lidar sees inside 1 m must already be triggering a stop, and the near-field check lives in the safety monitor, not in the perception pipeline that takes 100 ms. The Level 3 rule: the fastest, dumbest check gets the shortest path to the motors.

Failure: the 4G link drops. It will, several times a trip, under bridges and in dips. The robot must be fully able to drive without it: localisation, planning and the safety monitor all run on board. The link is only for the supervisor's decisions at hard spots (a road crossing, a blocked pavement). Rule: if the link has been silent for 10 s and the robot is at a decision point, it waits at the kerb. If it is mid-pavement it carries on to the next safe waiting spot. A supervisor watching 2 Mbit/s video over a 22 minute trip uses 330 MB of data, so the video is sent only at decision points and on request, not all the time.

Scale: one supervisor for many robots. A supervisor cannot drive robots: 300 ms round trip is 0.45 m of travel between a picture and a command. So the robot drives, and the supervisor only answers "may I cross?" and "which way round this van?". Each such question takes a human about 20 s. If one robot asks twice per trip, that is 40 s of attention per 22 minute trip, so in theory one supervisor can handle 33 robots. Questions cluster at busy times, so plan for 20 to 30. That number is the business case, and it follows straight from the latency arrow.

Step 6. Trade-offs

Choice	Chosen because	At the cost of
Walking speed, not cycling speed	stopping distance stays under 1 m, so a cheap short-range lidar is enough	22 minutes for 2 km, so fewer trips per day
All driving on board, supervisor only at decision points	the link is unreliable and slow, and safety must not depend on it	a more capable, more expensive computer on every robot
A pre-driven route map, not a general city map	localisation against a known route is far easier and more reliable	every new route needs a mapping drive first
Wait at the kerb when unsure	an idle robot is a small cost; a wrong crossing is a huge one	robots pile up at crossings on busy days; the supervisor needs a queue

Step 7. What I would build first

One robot, one route, a person walking behind it with a remote stop. No perception of people at all: the safety monitor stops the robot for anything inside 1 m and it waits. Measure how often it stops and for how long. Then add: the people detector so it can steer around rather than stop, then the supervisor console and the crossing question, then the second route.

Questions someone might push on, with answers

- *What about road crossings without a signal?* The robot never decides to cross on its own at first. It asks the supervisor, who looks at the video. Later, when the perception has months of data, crossing at a zebra crossing with no cars in view can be automated, and the supervisor is told rather than asked.
- *How does localisation survive GPS going to 20 m between tall buildings?* The Kalman filter weights each input by how sure it is. GPS gets a large uncertainty in the city and the cameras matching the route map take over. That is why the route is pre-driven.
- *Where is the BFS or A here?** The path planner. The route map is a grid, the robot's cell is the start, the next waypoint is the goal, and cells with people or kerbs are blocked. That is exactly tutorial/03-grids-bfs-dfs.md with a cost for "near a person".

5.4 A drone inspecting a wind turbine blade for cracks

Exercise 3 from Level 1. The request: "Design a drone that flies up to a wind turbine and photographs each blade closely enough to find cracks."

Step 1. What it must do and never do

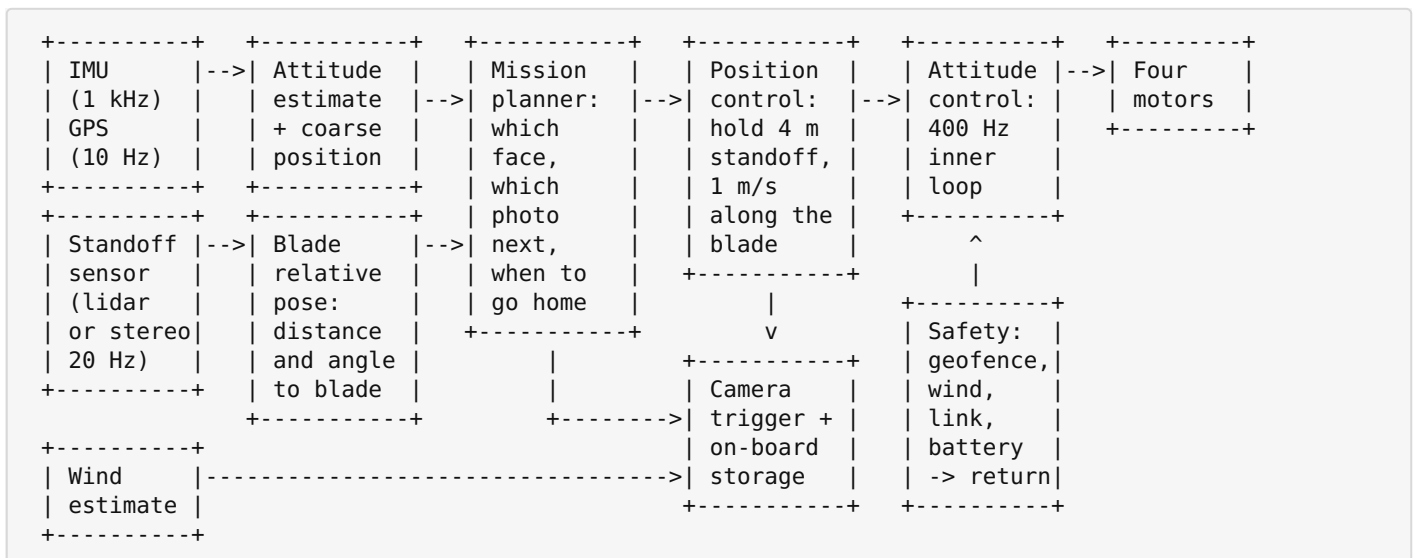
Who and where: an inspection team of two on a windy hill or on a boat, one turbine at a time, the turbine stopped and the blades parked. Done: every square metre of all three blades photographed at about 1 mm per pixel, cracks 3 mm and wider found, a report per blade. Never: touch the blade or tower, fly away (lose the link and keep going), fly in wind above its limit, lose the photos. Fixed: a commercial inspection drone with a 24 megapixel camera, 25 to 30 minutes of flight per battery. Today: a rope-access technician abseils the blade, which takes a day per turbine and is dangerous. A drone doing a turbine in under an hour is the target.

Step 2. Numbers first

Number	Value	Why it matters
Blade length	60 m; tower 100 m, so the tip is at 160 m	sets flight height and path length
Path	3 blades, 4 faces each (two sides, two edges), 720 m of flying	total time and photo count
Standoff	4 m from the blade	with a 24 mm lens on a full-frame sensor: 6 m by 4 m in frame, 1 mm per pixel
Photo spacing	70% overlap, so a new photo every 1.2 m; at 1 m/s that is 0.83 photos a second	600 photos per turbine
Storage	12 MB per photo, 7.2 GB per turbine	must be stored on board; 48 Mbit/s to stream it live, not available on a hill
Flight time	12 minutes of scanning plus 8 of transitions, 20 minutes	one battery per turbine, with no margin; plan for a swap
Wind	drone limit 12 m/s; gusts move it 0.5 m in a second	standoff must be held by a sensor, not by GPS

The number that decides the design is the last one. A gust that moves the drone 0.5 m towards a blade 4 m away is a near miss. The drone must measure its distance to the blade directly.

Step 3. Draw sense, think, act



Two loops, one inside the other. **Flight control** (attitude at 400 Hz, position at 50 Hz) keeps the drone level and where it is told to be. It exists in every drone and is usually bought, not written. **Mission planning** (1 Hz) decides where "where it is told to be" is: which face, how far along, when to trigger the camera, when to go home. Keep these in separate boxes and if possible on separate computers, because the mission code changes every week and the flight code must not.

Step 4. Label the arrows

Arrow	What travels	How often
IMU → Attitude estimate	turn rates and accelerations	1 kHz
GPS → Attitude estimate	position, good to 2 to 5 m, worse next to a steel tower	10 Hz

Arrow	What travels	How often
Standoff sensor → Blade relative pose	distance to the blade surface and its angle	20 Hz
Blade relative pose → Position control	"you are 4.3 m out, 5 degrees off square"	20 Hz
Mission planner → Position control	a target point along the face and a speed	1 Hz, held between updates
Mission planner → Camera trigger	"take a photo now" with the current pose	0.83 Hz
Camera → Storage	12 MB photo plus pose and time stamp	0.83 Hz, 10 MB/s to the card
Wind estimate → Mission planner and Safety	wind speed from how hard the motors work	1 Hz
Ground station → Mission planner	start, pause, abort, next blade	on change, a radio link
Safety → Position control	"return home now", overriding the mission	immediate

Step 5. The three hardest arrows

Latency and accuracy: the standoff sensor to position control. GPS cannot hold a 4 m gap next to a 100 m steel tower; it reflects the signal and the error grows to tens of metres. So the gap is held by a sensor pointed at the blade: a small lidar or a stereo pair, reading at 20 Hz. A gust pushes the drone 0.5 m in a second; at 20 Hz the control loop sees 0.025 m of that per reading and corrects. If the sensor loses the blade (looking past the tip, or at the sky), the position control must not go back to GPS and drift into the blade; it holds attitude, backs away 2 m, and asks the mission planner for a re-acquire.

Failure: the radio link or the battery. Every drone has a return-home rule; the question is what it returns *through*. From the far side of a blade at 160 m, straight home may go through the tower. The mission planner keeps a list of safe retreat points (out from the blade, then down) and the safety box uses them. Battery: 20 minutes of work against 25 to 30 of flight leaves no margin in wind, which raises the motor load. The rule is to return when the estimated time home plus 3 minutes exceeds the battery left, computed every second, not at a fixed percentage.

Scale: the photos. 600 photos, 7.2 GB, per turbine. Streaming them live would need 48 Mbit/s, which a hillside radio does not have. So the photos stay on the drone's card with the pose stamped on each, and are copied off between batteries. That means a lost drone is a lost turbine's work, and it means the report is built after landing, not in the air. The pose stamp is what lets the report say "crack at 23 m from the root on the pressure side".

Step 6. Trade-offs

Choice	Chosen because	At the cost of
Hold standoff with a blade-facing sensor, not GPS	not hitting the blade matters more than anything	another sensor, and a loop to write and test
4 m standoff with 1 mm per pixel	finds 3 mm cracks, and 4 m absorbs a 0.5 m gust	600 photos and 20 minutes; 2 m would be sharper but a gust could reach the blade
Photos on the card, report on the ground	the radio cannot carry 48 Mbit/s and the drone's compute is better spent on flying	no live crack detection; the team waits for landing
Bought flight controller, own mission planner	the flight loop is a solved problem and certified; the mission is where the value is	limited to what the flight controller's interface allows
Stop the turbine and park the blades	a moving blade tip at 80 m/s cannot be flown near	a turbine out of service for an hour; the operator must agree

Step 7. What I would build first

The drone with the bought flight controller, flown by hand along one face while the mission planner only triggers the camera and stamps the pose. Check the photos are sharp (1/1000 s exposure at 1 m/s is 1 mm of blur, one pixel, acceptable) and the overlap is right. Then add the standoff sensor and the hold loop, tested on a wall. Then the automatic face-by-face path. Wind and battery rules go in before the first turbine, not after.

Questions someone might push on, with answers

- *Why not one wide photo of the whole blade from far away?* At 60 m away the same camera gives 15 mm per pixel; a 3 mm crack is invisible. Resolution forces you close.
- *Why 70% overlap?* So every point is in at least three photos, which lets the ground software stitch the images and see a crack from three angles; it also covers a missed trigger.
- *What is the geofence?* A volume the drone must stay in: a cylinder around the turbine, plus a floor and ceiling. Leaving it triggers return-home regardless of the mission. This is the Level 4 idea of a simple check outside the clever code.

5.5 A warehouse with 200 grid robots

The request: "Design the system that runs 200 robots on top of a grid of storage bins, the way Ocado's grocery warehouses work." The robots are small cubes that drive on rails over a grid; below each cell is a stack of crates; a robot lowers a hook, lifts the top crate, and carries it to a pick station where a person or an arm takes the items out.

Step 1. What it must do and never do

Who and where: an indoor hall, no people on the grid, engineers can enter only when the fleet is stopped. Done: 12,000 crates delivered to pick stations an hour, orders complete on time. Never: two robots in the same cell, a robot driving off a stopped section, a stuck robot blocking a lane for long, a crate dropped. Fixed: the grid is 100 by 60 cells, cells are 1 m, robots move at 2 m/s, one wireless radio system. Today: a bigger warehouse with people walking to shelves; the grid is the replacement.

Step 2. Numbers first

Number	Value	Why it matters
Grid	100 by 60, 6000 cells, 1 m each	a path search over 6000 cells is tiny
Robot speed	2 m/s, so half a second per cell	the time slot the planner reserves per cell
Stopping	from 2 m/s at 4 m/s squared: 0.5 s, 0.5 m	a robot can stop inside one cell
Radio	each robot sends 50 bytes of status 10 times a second	200 robots is 100 kB/s, easy; the hard part is reliability, not volume
Battery	1 kWh, 300 W average draw, 3.3 hours running, 1 hour charging	23% of robots are charging at any time, so 260 robots for 200 working
Throughput	one crate per robot per minute	200 robots gives 12,000 an hour, the target

Step 3. Draw sense, think, act

CENTRAL (one computer, plus a hot standby)

EACH ROBOT (x200)

Step 4. Label the arrows

Step 5. The three hardest arrows

58 / 67

Scale: the central planner's cycle. 200 robots, each replanned about once a second, each A* over a few thousand (x, y, t) nodes taking about a millisecond: 200 ms of planning per second on one core. That fits, with room. If it did not, the fix is to plan only the robots whose plan changed, and to give each robot 10 cells rather than the whole route so a change far away does not touch it.

Latency and correctness: the reservation table. Every cell has a list of (robot, from time, to time). A robot moving at 2 m/s spends 0.5 s in a cell; the planner reserves 0.5 s plus a 0.2 s margin either side, to cover clock error and stopping. If a robot is late (a slow wheel, a heavy crate) its later reservations are wrong. So the planner re-checks reservations against reported positions every cycle and shifts the ones behind schedule, which may push other robots. Clocks matter: all robots sync to the central clock so "arrive at $t = 1042.3$ s" means the same thing to everyone.

Step 6. Trade-offs

The big one is central versus distributed.

Choice	Chosen because	At the cost of
Central planner with reservation table	global view gives no deadlocks and near-optimal traffic; 200 ms per cycle is affordable	a single point of failure, so a hot standby that mirrors the table, and a fleet that stops if both fail
Distributed (each robot plans, negotiates with neighbours)	no single point of failure; scales past 1000 robots	deadlocks in crowded corners are hard to prove absent; that is why it is not chosen here
Robots stop on 200 ms silence	never driving on a stale plan matters more than the throughput lost to radio blips	needs a very reliable radio; the hall gets many access points
Nearest idle robot for assignment first	simple, explainable, and half the possible gain comes from it	a smarter assignment later can raise throughput 10 to 20%
Charge at 20% battery, opportunistically at 40% when idle	a robot that dies on the grid blocks a lane and needs a person	260 robots bought to have 200 working

Step 7. What I would build first

A simulator: the grid, 20 robots, the reservation table, and the A* in (x, y, t). Prove no two robots ever share a cell, and measure crates per hour. Then five real robots on a real corner of the grid with the same central code, then the radio-silence stop rule tested by unplugging an access point. Only then 200.

Questions someone might push on, with answers

- *What if a robot breaks down on the grid?* The fleet monitor sees no progress for 5 s and marks the cell blocked; the planner routes around; a recovery robot tows it, or the section is stopped for a person. A stuck robot in a busy lane costs minutes, so the layout has more lanes than the minimum.
- *Why not put the reservation table on the robots?* Because 200 copies must agree, and agreement over a lossy radio is a research problem. One table, one clock, one truth, with a standby.
- *How is this the A in the tutorial?** Same code shape: a priority queue of nodes, a cost so far, a heuristic (Manhattan distance on a grid), and neighbours. The only changes are that a node carries a time, "wait here" is a neighbour, and a neighbour is skipped if the reservation table says the cell is taken at that time.

5.6 A surgical robot arm teleoperated by a surgeon

The request: "Design a robot arm that a surgeon controls from a console, the way CMR Surgical's Versius or Intuitive's da Vinci work." **Teleoperation** means a person moves a control and the robot copies the motion; the robot decides nothing about where to go.

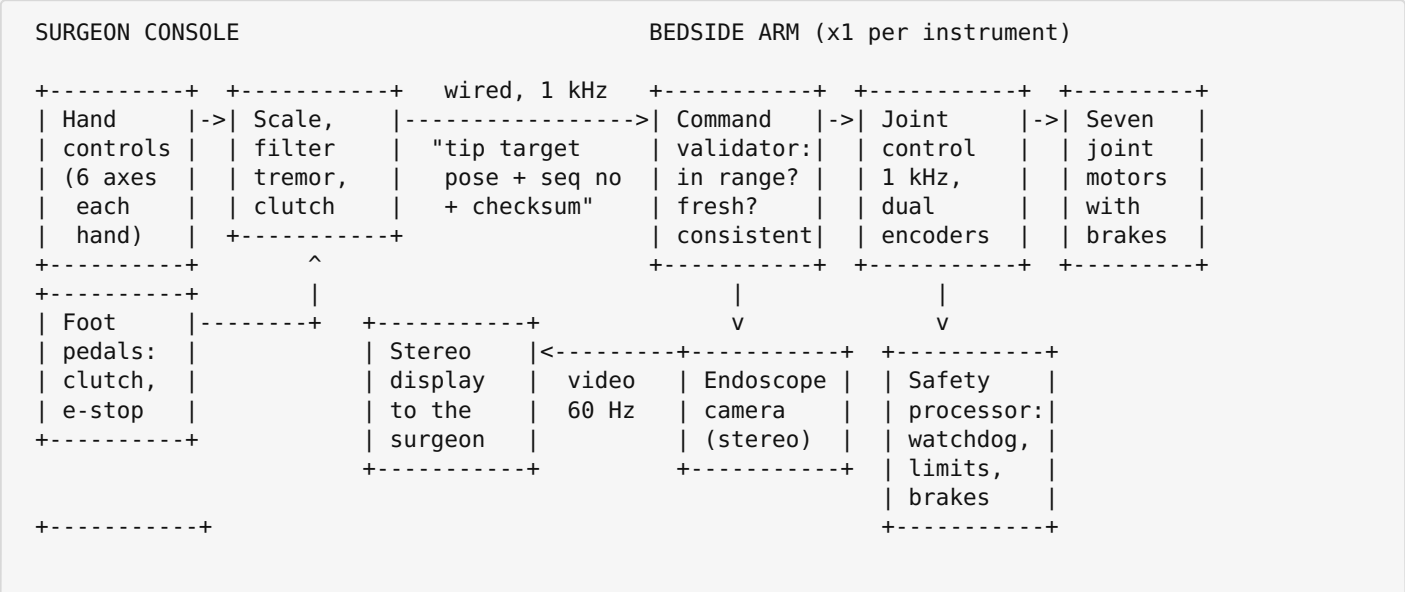
Step 1. What it must do and never do

Who and where: an operating theatre. The surgeon sits at a console a few metres from the patient; a nurse stands by the arm. Done: the instrument tip follows the surgeon's hand, scaled down and steadied, with a stereo (two-eye) video view that feels live. Never: move when the surgeon has not moved, move further or faster than commanded, keep moving after a fault, lose the record of what happened. Fixed: a seven-joint arm holding a thin instrument through a small port in the body; a wired network in the room, no internet in the loop. Today: the surgeon holds long instruments by hand; the robot adds scaling, tremor removal and a better view. It adds no autonomy at all, and that is a requirement, not a limitation.

Step 2. Numbers first

Number	Value	Why it matters
Command rate	1 kHz from console to arm	1 ms per step, so a stop is felt within a few ms
Motion scaling	3 to 1: a 3 cm hand move is a 1 cm tip move	hand at 10 cm/s gives a tip at 3.3 cm/s, 33 micrometres per 1 ms cycle
Tremor	human hand tremor is 8 to 12 Hz; filter cuts above 6 Hz	a 0.2 mm tremor becomes 0.07 mm at the tip and is then filtered
Video path	camera frame 16.7 ms (60 Hz) + encode 10 ms + display 16 ms, about 43 ms	the surgeon feels lag above roughly 100 ms; 43 ms leaves margin
Command path	sample 1 ms + wired network 1 ms + control 1 ms, 3 ms	the video is the slow side, not the commands
Watchdog	freeze if no valid command for 20 ms	the tip moves at most 0.67 mm before freezing at full speed
Logging	7 joints, 4 bytes each, 1 kHz: 28 kB/s, 400 MB per 4-hour operation; video 2 streams at 8 Mbit/s is 29 GB	both must be kept, for years

Step 3. Draw sense, think, act



```
| Audit log |<---- every command, every joint reading, every fault, both video streams
+-----+
```

The **clutch** is a pedal that disconnects hand from tip so the surgeon can reposition the hands, like lifting a mouse. The **command validator** checks each message: a sequence number so nothing is repeated or skipped, a checksum so nothing is corrupted, and a range check so a tip target is never outside the allowed volume or further than 1 ms of motion from the last one. The **safety processor** is a second, simpler computer that reads the same encoders and can apply the brakes on its own.

Step 4. Label the arrows

Arrow	What travels	How often
Hand controls → Scale and filter	position and orientation of each hand, 6 numbers	1 kHz
Foot pedals → everything	clutch on/off, e-stop	immediate, wired
Console → Command validator	tip target pose, sequence number, checksum	1 kHz over a dedicated wire
Command validator → Joint control	a validated tip target, or "hold"	1 kHz
Joint control → Motors	a current per joint	1 kHz
Encoders → Joint control and Safety processor	two independent angle readings per joint	1 kHz, both boxes read both
Endoscope → Display	two 1080p video frames	60 Hz, encoded, under 43 ms end to end
Safety processor → Brakes	"engage"	immediate, a physical line
Everything → Audit log	commands, joint states, faults, video	1 kHz for data, continuous for video

Step 5. The three hardest arrows

Latency: video to the surgeon's eyes. The commands are fast (3 ms). What the surgeon feels is the video: 43 ms in the budget above. Every extra encode step, every Wi-Fi hop, every frame buffer adds to it, and past about 100 ms surgeons overshoot and slow down. So the video path is wired, uncompressed where possible, and measured on every build with a flashing light in front of the camera and a photodiode on the display.

Failure: any single part. The rule is that no single failure may cause motion. Each joint has two encoders (one absolute, one incremental); if they disagree by more than a small angle the safety processor brakes. Two controllers run the same code in lockstep and compare outputs. Power is dual. The watchdog freezes the arm 20 ms after the last valid command, so a cut cable means the tip moves 0.67 mm at most, and then holds still. Holding still, not retracting, is the safe state: a retract could tear tissue. The surgeon or nurse then removes the instrument by hand.

Correctness: the audit log. Every command and every joint reading at 1 kHz, 400 MB per operation, plus 29 GB of video, written to two disks, time-stamped from one clock, kept for years. If something goes wrong in theatre, this is the only way to know whether the robot did what it was told. It also feeds the regulator. The log writer must never slow the control loop, so it reads from a buffer the loop fills and the loop never waits for the disk.

Step 6. Trade-offs

Choice	Chosen because	At the cost of
No autonomy at all	the surgeon is responsible and the regulator certifies a tool, not a decision maker	no "auto-suture", however tempting; the robot cannot be cleverer than its operator
Freeze, not retract, on fault	staying still cannot cause new injury; moving might	someone must remove the instrument by hand after a fault
Wired, dedicated links in the room	latency and reliability are certain; nothing shares the cable	no remote surgery over a network, which is a different, harder product
Dual encoders, dual controllers, separate safety processor	a single failure must never move the arm	roughly twice the electronics, and a lot of testing
3 to 1 scaling with a 6 Hz filter	precision matters more than speed for fine work	large moves need the clutch several times; surgeons switch scaling for coarse moves

Step 7. What I would build first

The command path with the validator and watchdog, on a bench, driving one joint, with a switch that cuts the cable. Prove the freeze is under 20 ms every time, thousands of times. Then the second encoder and the safety processor. Then the video path with a latency measurement. The audit log is in from day one because it is how you test everything else.

Questions someone might push on, with answers

- *Why not add a "no-go zone" so the tip cannot touch an organ?* That is autonomy: the robot overriding the surgeon. It also needs the robot to know where the organ is. Products do add simple bounds set by the surgeon before the case; that is a range check, not a decision.
- *Would you use ROS 2?* Not in the control or safety path. Certified medical software needs every line accounted for, and the 1 kHz loop needs guaranteed timing. ROS 2 would be fine for non-critical tooling: setup screens, the log viewer, simulation.
- *How is this different from the pick-and-place arm in 5.2?* Same joint loop, same idea of a separate safety box. The difference is who decides: 5.2 has a perception and planning stack that decides; this has a human, and the whole design exists to carry the human's intent faithfully and to prove afterwards that it did.

5.7 A self-driving shuttle on a fixed route

The request: "Design a small passenger shuttle that drives itself around a fixed 5 km loop with six stops, the kind Oxa and Aurigo have trialled in UK towns and airports."

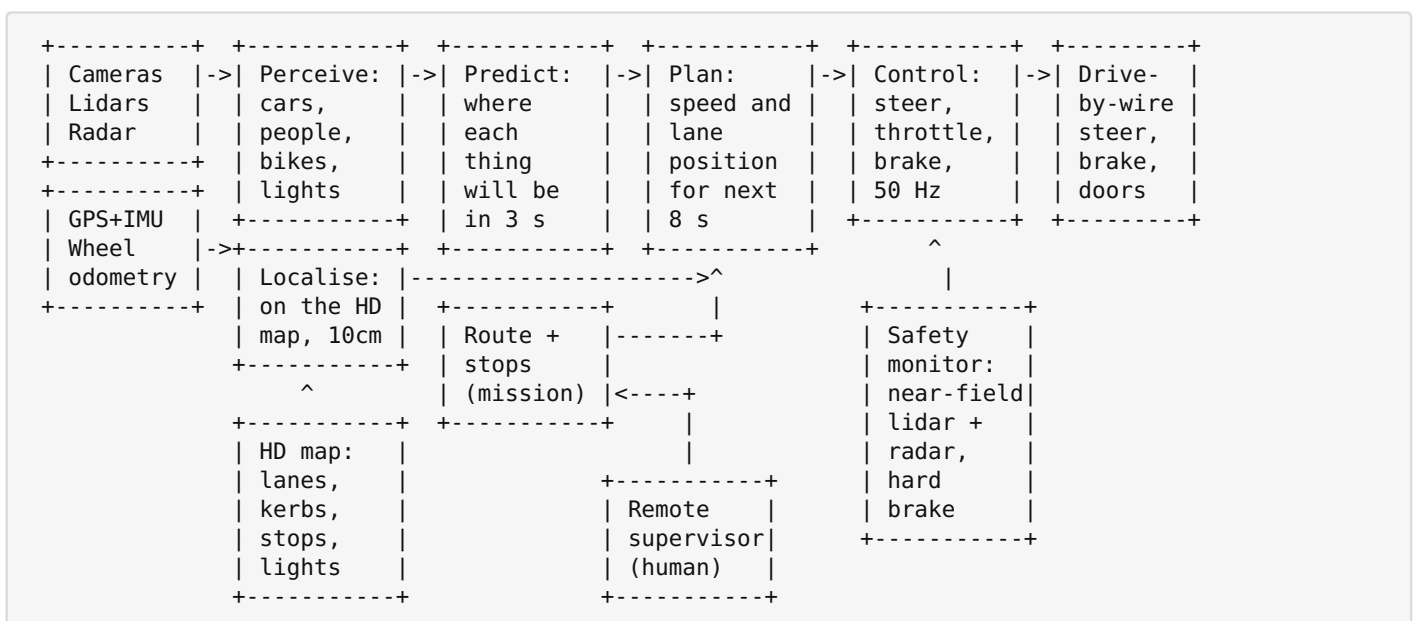
Step 1. What it must do and never do

Who and where: public roads at low speed, mixed with cars, cyclists and pedestrians; up to eight passengers. Done: the loop every 15 minutes, all day, stopping at each stop, at 25 km/h. Never: hit anything, run a red light, leave the route, move with a door open, strand passengers with no way to get help. Fixed: the route, which can be mapped in detail in advance; a vehicle with drive-by-wire (the computer can steer, brake and accelerate through electrical commands); a mobile network. Today: a bus with a driver. The question of whether a safety driver sits on board is answered in Step 6.

Step 2. Numbers first

Number	Value	Why it matters
Speed	25 km/h, 6.9 m/s	stopping distance and how far it moves during any delay
Loop	5 km, 12 minutes driving plus 6 stops of 30 s: 15 minutes, 4 trips an hour	the timetable
Stopping	hard brake 3 m/s squared: 8 m; plus 0.3 s of latency, 2.1 m: 10 m. Comfortable 2 m/s squared: 14 m	the shuttle must reliably see and classify things at 30 m, which it reaches in 4.3 s
Pedestrian prediction	3 s ahead: a walker at 1.5 m/s moves 4.5 m, a runner at 4 m/s moves 12 m	the size of the "might be there" zone in front of a person
Sensors	4 cameras 1080p at 20 Hz: 500 MB/s raw; 2 lidars of 32 beams: 1.3 million points a second each, 42 MB/s together	all on board; nothing raw leaves the vehicle
Localisation	20 Hz, error under 10 cm against the HD map	needed to stay in lane and stop at the kerb
Remote link	100 ms typical, sometimes seconds; the shuttle moves 0.7 m in 100 ms, 3.5 m in 500 ms	a remote person can advise, never steer

Step 3. Draw sense, think, act



The **HD map** (high definition map) is a detailed map of the route, made in advance: every lane edge, kerb, stop line, traffic light position, and the shape of the buildings and signs the lidar will see. **Localise** matches the live lidar against that shape to find the vehicle to 10 cm; GPS alone is metres out. **Predict** turns "a person at (x, y) moving at 1.5 m/s" into "a person somewhere in this 4.5 m long region over the next 3 s", and the planner keeps out of that region. **Radar** measures distance and speed directly and works in rain when cameras and lidar suffer.

Step 4. Label the arrows

Arrow	What travels	How often
Cameras → Perceive	four 1080p images	20 Hz

Arrow	What travels	How often
Lidars → Perceive, Localise, Safety monitor	point clouds	10 to 20 Hz
HD map → Localise and Plan	the map section around the vehicle, about 10 MB for the loop	loaded once, queried continuously
Localise → Plan and Predict	pose to 10 cm, with a "how sure"	20 Hz
Perceive → Predict	a list of objects: type, position, speed, size	20 Hz
Predict → Plan	for each object, the region it may occupy over 3 s	20 Hz
Mission → Plan	the next stop and the lane to follow	on change
Plan → Control	a trajectory: positions and speeds for the next 8 s	20 Hz
Control → Drive-by-wire	steering angle, throttle, brake	50 Hz
Shuttle → Remote supervisor	compressed video from one camera 2 to 4 Mbit/s, pose, state, and "I need a decision"	video on demand, state 1 Hz
Remote supervisor → Mission	"proceed", "wait", "use the other lane", "stop at next safe place"	on change
Safety monitor → Drive-by-wire	"brake now"	immediate, independent of everything above

Step 5. The three hardest arrows

Latency: something in front to the brakes. From 6.9 m/s the vehicle needs 10 m to stop hard, 14 m comfortably. The perceive, predict, plan chain takes 0.3 s and 2 m of travel; that is already in the 10 m. The safety monitor bypasses the chain: lidar and radar returns inside a box in front of the vehicle, scaled by speed, trigger the brake directly, with no classification. It brakes for a plastic bag too. That is accepted, because braking for a bag is embarrassing and hitting a child is not.

Failure: the remote supervisor and the link. A stuck shuttle (a parked van across the lane, an unclear roadworks layout) asks the supervisor. The supervisor sees the video and answers a question from a short list: proceed, wait, take the other lane, stop safely. The supervisor never steers, because at 100 ms to seconds of delay the vehicle has moved metres. If the link is down the shuttle waits in lane with hazard lights and passengers are told; it never guesses. If it is somewhere unsafe to wait it crawls to the next mapped safe stop.

Scale: the perception data and the map. 500 MB/s of camera data never leaves the vehicle; it is processed on board and only objects, at a few kB per frame, go to prediction. The HD map for a 5 km loop is small, about 10 MB, but it goes stale: roadworks move kerbs and cones. So the vehicle compares what it sees with the map every trip, and a mismatch (a kerb where there was none) both slows the vehicle and flags the map for an update by the mapping team.

Step 6. Trade-offs

Choice	Chosen because	At the cost of
Fixed route with an HD map	localisation to 10 cm and a known set of hazards; "anywhere" is a far harder product	every route change needs a mapping drive and a review
25 km/h, not 50	10 m stopping makes 30 m of reliable sensing enough; the safety case is provable	slower than a bus; only fits short urban loops
Safety monitor brakes for anything	nothing must depend on classification being right	false stops for bags, birds and heavy rain; passengers feel them

Choice	Chosen because	At the cost of
Remote supervisor advises, never drives	link delay makes driving unsafe; advice at decision points is safe	the shuttle waits when it needs a human and none is free
A safety driver on board during trials	the safety case is built from evidence, and the driver provides both a fallback and the evidence; UK trials have run this way while the law (the Automated Vehicles Act 2024) sets the route to removing the driver	the cost of a driver undoes the business case, so the design must record everything needed to justify removing them: every takeover, why, and what the system would have done

Step 7. What I would build first

The vehicle with the safety monitor and drive-by-wire only, driven by a person on the route, recording all sensors. That gives the map and a log to test perception against offline. Then localisation on the map, checked against the human drive. Then the full chain, first in replay, then on a closed road, then on the route with a safety driver. Takeovers are counted from the first day, because the number of miles between takeovers is the number that decides when the driver can leave.

Questions someone might push on, with answers

- *What about traffic lights?* The map says where each light is, so the camera looks in a small known region of the image, which is far more reliable than finding lights anywhere. Where possible the light also sends its state over a short-range radio, and the two must agree, or the vehicle stops.
- *What happens in fog?* Radar keeps working, cameras and lidar lose range. The planner reduces speed so that stopping distance fits inside what the sensors can still see, and below a threshold the shuttle goes to the next stop and waits. That is the same rule as the drone's wind limit in 5.4.
- *Why not a bigger, faster vehicle?* Stopping distance grows with the square of speed. At 50 km/h the hard stop is 32 m and the sensing requirement roughly doubles; the safety case becomes a different project.
- *Where does A or Dijkstra appear?** In the mission box, over the route graph (stops and junctions as nodes), the Dijkstra in tutorial/04-graphs-dijkstra.md. The lane-level planner is a different tool: it searches over short trajectories, not grid cells.

5.8 Common mistakes and how to avoid them

- **Starting with technology instead of requirements.** "I would use ROS 2 and a Jetson" before knowing the speed or the space. Fix: five questions, five numbers, then boxes. The technology falls out of the numbers.
- **Unlabelled arrows.** A line between two boxes with nothing on it is a question you have not answered. Fix: what travels, how often, on every arrow, even if you have to guess.
- **No numbers.** "Fast", "real-time", "lots of data". Fix: a number with a unit, rough is fine. 1.5 m/s, 20 Hz, 500 MB/s. Then check one of them against another: does the stopping distance fit inside the sensor range?
- **No failure story.** A design where every box works. Fix: for the three hardest arrows, say what happens when the arrow stops, and what the robot does about it.
- **Putting safety in the cloud, or in the clever code.** If the stop depends on the network or on perception being right, it is not a stop. Fix: a simple safety box on the robot with its own sensor and a wired line to the motors (5.2, 5.3, 5.6, 5.7 all have one).
- **Remote driving over a mobile link.** 300 ms is metres of travel. Fix: the robot drives, the remote human answers questions at decision points (5.3, 5.7).

- **One giant box called "AI".** Fix: split think into perceive, localise, predict, plan, control, each with a one-sentence job and an interface to the next.
 - **Forgetting the boring loops.** Battery, charging, logging, the map going stale, clock sync. Fix: 5.5's charging arithmetic (260 robots for 200 working) and 5.6's audit log are design decisions, not afterthoughts.
 - **Trade-offs stated as facts.** "Central planning is better." Fix: "Central because global optimality matters more than the single point of failure here, which we cover with a standby". Every choice has a cost; say it.
 - **No first step.** A design that needs everything before anything works. Fix: name the thinnest thing that tests the riskiest arrow, and build that (every Step 7 above).
-

5.9 A question bank

Fifteen more prompts, easiest first. For each, run the seven steps on paper in 40 minutes, then compare your hardest arrows and trade-offs with the six worked designs above.

1. A robot lawnmower for a garden with a pond.
 2. A window-cleaning robot for a three-storey office.
 3. A robot that restocks shelves in a supermarket at night.
 4. A robot that carries linen between floors in a hospital, using the lifts.
 5. A drone that counts sheep on a hillside farm.
 6. A robot arm that loads and unloads a CNC machine in a small workshop.
 7. A weeding robot that drives along crop rows and pulls weeds with a small arm.
 8. A fleet of ten cleaning robots in an airport terminal, sharing one map and one charging bay.
 9. An underwater robot that inspects the legs of an offshore platform, with no GPS and a slow acoustic link.
 10. A robot that sorts parcels by reading their labels on a conveyor, at two parcels a second.
 11. A quadruped robot that carries tools across a construction site with no flat floors.
 12. A humanoid that unloads boxes from a lorry, working next to people.
 13. A fleet of 50 agricultural robots on farms across a region, with one remote support team.
 14. A robotic taxi in a city centre with no fixed route and no safety driver.
 15. A robot that assists a surgeon by holding the endoscope and following the instruments, with the minimum autonomy needed to be useful and no more.
-

5.10 In your own words

A design question is a vague request, and the skill is turning it into requirements, numbers, boxes, labelled arrows, three hard arrows gone deep, trade-offs said as "X because Y matters more than Z", and a first thing to build. The same seven steps produced six very different robots above because the numbers were different: 1 m/s and a person's hand, 1.5 m/s and a 300 ms link, a 0.5 m gust against a 4 m gap, 200 robots and one clock, a surgeon's 43 ms of video, a shuttle's 10 m to stop. Safety always ended up in a small dumb box on the robot. The remote human always ended up answering questions, never steering.

Check yourself

1. What are the five questions to ask before drawing anything, and why does "what happens today without the robot" belong on the list?
2. Which five numbers do you write down first, and give one example from this level of a number that decided a design.

3. In 5.2, why is the light curtain 0.9 m outside the arm's reach and not 0.35 m?
 4. In 5.3, why can the remote supervisor not drive the robot, and what is the one number that says so?
 5. In 5.4, why is GPS not used to hold the 4 m standoff, and what is used instead?
 6. In 5.5, what single rule on each robot makes it safe to trust a central planner, and what does that rule cost?
 7. In 5.6, why is the safe state "freeze" rather than "retract", and how far can the tip move before the watchdog acts?
 8. In 5.7, what does the safety monitor brake for, and why is that accepted?
 9. State the central versus distributed planner trade-off in one "X because Y matters more than Z" sentence, for the warehouse in 5.5.
 10. Pick any prompt from 5.9. Without looking back, name its three hardest arrows and what you would build first.
-

5.11 Try it

Take prompt 8 from the bank (ten cleaning robots in an airport) and write the full seven steps in the same format as 5.2 to 5.7: five answers, five numbers, an ASCII diagram, an interface table, three hard arrows, a trade-off table, what to build first, and three questions someone might push on. Check every number you write with a few lines of Python, the way the numbers in this level were checked. When you have done it, do it again for prompt 14, and notice which arrows are the same and which are new.